

uc3m | Universidad **Carlos III** de Madrid

Master Degree in Cybersecurity

Academic Year: 2024-2025

Master Thesis “The Game Beneath the Game:
The Evolution of Cheats and Anti-Cheat
Systems in Counter-Strike 2”

Pablo Valiente

Tutor: Antonio Nappa

Universidad Carlos III de Madrid

Date: June 2025

Abstract

This thesis examines the phenomenon of cheating in online multiplayer games through a dual approach: a conceptual study and a practical implementation.

The first part offers a theoretical exploration of cheating as a socio-technical and historical process. It begins by reflecting on the inherently competitive nature of human behavior and considers how advances in computing and online infrastructure have enabled new forms of digital confrontation. Cheating is framed not as an isolated anomaly, but as a consequence of structural dynamics within highly competitive virtual ecosystems. The study also traces the evolution of anti-cheat technologies and the emergence of a **commercially driven underground industry**, emphasizing the limitations of current detection mechanisms and their broader implications for player trust and platform integrity.

The second part adopts an applied perspective. It presents the design and development of a custom cheat for **Counter-Strike 2**, used as a case study to evaluate the resilience of contemporary anti-cheat systems, with a particular focus on Valve's **VAC**. This practical component enables the identification of architectural weaknesses, the assessment of real-world detection efficacy, and a reflection on the technical and ethical complexities of securing competitive environments.

Dual Perspective as a Core Contribution: This thesis stands out by offering a **comprehensive dual approach that integrates both offensive and defensive perspectives**. On the one hand, it thoroughly dissects how cheats are engineered and deployed in real-world scenarios. On the other, it critically analyzes the structural and technical limitations of current anti-cheat mechanisms. This dual lens not only enriches the academic discourse on game security but also provides actionable insights for both exploit mitigation and defense design.

The research as a whole articulates cheating in video games as both a *technological* and *cultural* issue, grounded in theory, examined through practice, and situated within a broader discourse on digital integrity. The thesis concludes by outlining strategic recommendations for more adaptive, transparent, and ethically robust anti-cheat frameworks.

Keywords: Cheating, Online Games, Counter-Strike 2, Anti-Cheat Systems, Digital Competition, Game Security, Applied Analysis

DEDICATION

CONTENTS

1	Introduction	2
2	From Primitive Survival to Digital Domination	6
1	The Competitive Nature of Humanity	6
2	The Birth of Computation and the Internet: A New Digital Battleground	7
2.1	Early Computational Tools: From the Abacus to Mechanical Calculators	7
2.2	The 19th-Century Revolution: The Birth of Programmable Machines	7
2.3	The Acceleration of Computation in the 20th Century	8
2.3.1	World War II and the Role of Computation	8
2.3.2	Post-War Expansion: From Military to Commercial Computing	8
2.4	The Birth of the Internet: From Military Networks to Global Infrastructure	8
2.5	The Digital Arena: From Computing to Competition	9
2.6	From Networks to Virtual Battlefields	9
3	The Evolution of Cheating: From Debugging Tools to an Organized Industry	11
1	The Early Days: Debugging Tools and Experimental Modifications	11
2	The Rise of Cheating in Consoles and PC	11
2.1	Cheat Devices: Game Genie, GameShark, and Pro Action Replay	12
3	The Transition to Online Gaming: Cheating Becomes a Competitive Threat	13
4	The Birth and Evolution of Anti-Cheat Systems	13
5	The Commercialization of Cheating	16
4	The Prevalence and Impact of Cheating in Online Multiplayer Games	19
1	Cheating as a Critical Threat to Player Retention	19
2	Case Study: Cheating in <i>Counter-Strike 2</i>	20
2.1	Trust Factor and Reporting System Limitations	21
2.2	Economic and Psychological Impact	21
3	Community Perception and the Culture of Cynicism	22
4	Legal and Ethical Responses to Cheat Development	22
5	Anti-Cheat Technologies: Current Trends and Limitations	23

5	Injection and Detection Techniques	24
1	The Role of DLL Injection in Cheat Development	24
2	Overview of Code Injection Techniques	25
3	Detection Techniques in Anti-Cheat Systems	27
4	Extended VAC Capabilities and Developer Countermeasures	29
6	Hooking and Reversing: Core Techniques of Our Cheat Framework	32
1	Reverse Engineering and Function Discovery	32
2	Why Trampoline Hooks Are Effective	33
3	Implementing Hooks with MinHook	33
4	Final Thoughts on Trampoline Hooking	35
7	Internal Game Structure: Logic Behind Counter-Strike 2	36
1	Understanding Entity Lists in Video Games	36
1.1	Common Structures for Entity Lists	37
2	Entity Organization in Counter-Strike 2	37
3	Entity List Traversal	40
3.1	Locating and Identifying the Entity List in Memory	40
3.2	Reverse Engineering the Engine's Entity Resolution Logic	42
3.2.1	Efficient Entity Tracking: Hooking OnAddEntity and OnRemoveEntity	43
8	Cheat Features Engineered for Counter-Strike 2	48
1	Chams: Enhancing Visibility Through Engine-Based Rendering	49
1.1	Extracting Default Materials from the Game Engine	50
1.2	Generating Custom Materials	51
1.3	Hooking CreateMaterial and Injecting Custom Materials	52
1.4	Loading KV3 Materials into the Game	53
1.5	Hooking OnDrawModelExecute and Overriding Materials	54
1.5.1	Filtering Target Entities	55
1.5.2	Overriding Material Pointers	56
1.5.3	Chams Preview	57
1.5.4	Chams on Local Player and Weapon Models	60
1.5.5	Wall Visibility and Through-Wall Chams	62
1.5.6	Configuration Menu	63
2	Aimbot: Automated Aim to Enemy Players	65
2.1	Computing Aiming Angles	65
2.1.1	Deriving the Direction Vector	65
2.1.2	Computing the Player's Position	66
2.1.3	Centering Coordinates on the Enemy	66
2.2	Local Coordinate System and Angle Calculation	67
2.2.1	Mathematical Computation of Aiming Angles	67
2.3	Overwriting Player View Angles	69

2.4	Smoothing for Stealth	69
3	Bone Calculation: Extracting and Rendering Player Skeletons	70
3.1	Extracting Bone Data from Player Entities	70
3.2	Visualizing Bone Indices on the Player Model	71
3.3	Rendering a Connected Skeleton	72
4	Anti-Aim: Misleading Enemy Targeting Systems	74
4.1	What Is Anti-Aim?	75
4.2	Types of Anti-Aim	75
4.3	Lower Body Yaw (LBY) and the LBY Breaker	75
4.4	Benefits for the Cheater	76
4.5	Importance in Modern Cheats	76
9	Synthesis and Closing Thoughts	78
	Project Timeline (Gantt Chart)	81
	Bibliography	83

LIST OF FIGURES

2.1	LAN party in the late 1990s, a grassroots precursor to modern esports.	9
3.1	Game Genie	12
3.2	Chronological overview of key anti-cheat systems introduced between 2000 and 2023.	15
4.1	Evolution of Average CS2 Player Counts (January 2024 to May 2025). Source: SteamCharts.	20
5.1	Simplified flowchart of dynamic module injection using Manual Mapping.	27
5.2	Example of VMT Hooking.	29
7.1	Entity Hooking Flow: Interception and Redirection of OnAddEntity Function	47
8.1	Chams applied to an enemy player using the Glow material.	52
8.2	Example of visible and invisible Chams applied to an enemy model & to a Weapon.	56
8.3	Illuminated Yellow Chams, highly visible and reflective under all lighting conditions.	57
8.4	Flat Pink Invisible Chams, used to indicate models occluded by geometry.	57
8.5	Glow Yellow, produces a bloom effect around silhouettes.	58
8.6	Flat Pink Visible, standard colored overlay visible only when in line of sight.	58
8.7	Purple Fresnel, implements rim lighting using Fresnel falloff for edge highlights.	59
8.8	Green Plastic, glossy effect mimicking polymer reflectivity.	59
8.9	High Intensity Glow, amplifies model brightness for high-visibility targeting.	60
8.10	Chams override on local weapon (pink AK-47) and player hands (blue).	60
8.11	Alternate override combination using the same filtering logic.	61
8.12	Hands and weapon with different Chams styles, applied simultaneously.	61
8.13	Final variation of applied materials to local player assets.	62
8.14	Chams without wall visibility. Enemy is fully occluded by geometry.	62
8.15	Chams with wall visibility enabled. The enemy becomes visible despite occlusion.	63
8.16	Main Chams menu with per-entity toggles.	63
8.17	Color picker UI for selecting Chams tint.	64

8.18	Weapon and arms material override section.	64
8.19	Representation of the player and enemy positions on the map.	66
8.20	Original view with coordinate axes.	67
8.21	3D representation of the aiming direction vector.	68
8.22	Visualizing all bone indices rendered over the player model.	71
8.23	Skeleton & Bone Id.	73
8.24	Skeleton & Bloom Orange Chams.	73
8.25	Skeleton in a real game.	74
8.26	Skeleton & Chams in a real game.	74

LIST OF TABLES

- 3.1 Overview of major CS2 cheat providers, including release year, pricing models, and access restrictions. 17
- 4.1 Survey Results on Cheating in Online Games (Irdeto, 2018) 19
- 4.2 Activity of Banned Accounts Prior to Enforcement (January to April 2024) 21
- 8.1 Overview of Core Cheat Features 48

1. INTRODUCTION

Context and Motivation

The transformation of video games from a niche pastime into a global cultural and economic phenomenon has fundamentally altered the ways in which individuals interact, compete, and even pursue professional careers. In the era of **competitive gaming**, players participate in high-stakes environments where success hinges on precision, strategic thinking, and rapid reflexes. Driven by global connectivity and high-speed internet, on-line multiplayer games have evolved into structured **competitive ecosystems** featuring ranking systems, seasonal rewards, and financial incentives.

Today, video games function not only as a form of entertainment but also as **social platforms**, **economic engines**, and **professional arenas**. However, this rapid evolution has brought with it a persistent and multifaceted threat: **cheating**.

Through unauthorized software modifications, certain players seek to gain unfair advantages that disrupt the integrity of competition. These interventions range from memory manipulation and artificial input assistance to hardware-based exploits and network-level interference. As a result, an ongoing **arms race** has emerged, pitting developers, who design increasingly sophisticated anti-cheat mechanisms, against cheat creators, who constantly innovate to circumvent detection.

Moreover, the deployment of modern anti-cheat technologies raises a series of complex **legal**, **ethical**, and **environmental** concerns. Effective detection often requires deep access to system resources, prompting significant debates around user privacy and digital rights. At the same time, the commercialization of cheat software causes substantial economic harm to developers and challenges regulatory frameworks on a global scale. Additionally, the computational burden imposed by advanced detection mechanisms contributes to a growing environmental footprint within digital infrastructures.

To transcend theoretical discourse, this thesis adopts an applied methodology: the design, development, and evaluation of a custom-built cheat for a widely played multiplayer game.

Objectives and Scope

The primary aim of this thesis is to investigate the phenomenon of cheating in competitive video games through a dual lens: theoretical inquiry and hands-on experimentation. The research is articulated around three core objectives:

1. **To analyze the historical evolution of cheating in video games**, and to assess its influence on game design, player behavior, and the industry's countermeasures.
2. **To examine and categorize common cheating techniques**, including software manipulation, visual enhancements, and attacks, in terms of both technical function and competitive impact.
3. **To implement and evaluate a custom cheat in a contemporary multiplayer title**, using this case study to test the efficacy of current anti-cheat mechanisms and identify systemic weaknesses.

Competencies and Alignment with the Master's Program

This work is directly aligned with the core learning outcomes of the **Master's Degree in Cybersecurity**, particularly in the domains of **reverse engineering**, **intrusion analysis**, and **digital forensics**. Throughout the development of the project, the following competencies will be applied and demonstrated:

Basic Competencies

- **[CB6]** Conduct original research into the security challenges associated with competitive gaming, focusing on cheating and related countermeasures.
- **[CB7]** Apply cybersecurity techniques to the context of video game security, including the detection, development, and circumvention of unauthorized interventions.
- **[CB8]** Evaluate the legal and ethical dimensions of cheat creation and distribution in light of current digital legislation.

General Competencies

- **[CG1]** Analyze real-world intrusion scenarios to identify technical vulnerabilities in online gaming environments.
- **[CG2]** Understand and assess the internal architecture of anti-cheat systems, evaluating their effectiveness and limitations.
- **[CG3]** Produce clear and rigorous technical documentation that reflects expertise in offensive and defensive security techniques.

Specific Competencies

- [CE1] Understand the psychological and sociological factors that motivate cheating in competitive contexts.
- [CE2] Analyze advanced evasion strategies used by cheat developers and their implications for detection models.
- [CE3] Monitor and evaluate emerging threats in the field of game security as a subset of broader cybersecurity challenges.
- [CE4] Conduct forensic investigations into game memory and process manipulation to detect unauthorized behavior.
- [CE6] Critically assess the internal logic and performance trade-offs involved in the implementation of modern anti-cheat technologies.

Document Structure

The remainder of this thesis is organized as follows:

- **Chapter 1: Introduction** – Presents the motivation, objectives, competencies, and structural overview of the research.
- **Chapter 2: From Primitive Survival to Digital Domination** – Explores the human drive for competition and its historical transition into virtual environments, with emphasis on the technological developments that enabled competitive gaming.
- **Chapter 3: The Evolution of Cheating: From Debugging Tools to an Organized Industry** – Chronicles the emergence of cheating practices, their evolution into a commercial ecosystem, and the industry’s reactive development of anti-cheat systems.
- **Chapter 4: The Prevalence and Impact of Cheating in Online Multiplayer Games** – Analyzes the effects of cheating on gameplay integrity, user experience, and industry revenue, with a specific focus on the game **Counter-Strike 2**.
- **Chapter 5: Injection and Detection Techniques** – Provides an overview of the technical mechanisms behind code injection, cheat detection, and the adversarial dynamics between attackers and defenders.
- **Chapter 6: Internal Game Structure: Logic Behind Counter-Strike 2** – Investigates the architectural design of CS2’s engine, including memory organization, entity management, and reverse engineering techniques.

- **Chapter 7: Cheat Features Implemented in Our Cheat for CS2** – Describes the practical implementation of various cheat modules and evaluates their effectiveness against contemporary anti-cheat systems.
-

2. FROM PRIMITIVE SURVIVAL TO DIGITAL DOMINATION

1 The Competitive Nature of Humanity

The pursuit of **superiority, recognition, and dominance** has been a defining trait of human behavior across history. From primitive survival strategies to contemporary technological landscapes, the drive to **outperform others** has consistently propelled advancements in science, warfare, politics, and more recently, in digital environments.

This impulse extends beyond ambition; it constitutes a **deep-seated psychological mechanism** that shapes social dynamics and personal identity. According to Festinger [1], individuals tend to evaluate themselves through comparison with others, reinforcing competition as a means of attaining status and self-validation. Similarly, McClelland [2] demonstrated through his **Achievement Motivation Theory** that the aspiration to succeed and surpass peers is central not only to individual fulfillment but also to broader social hierarchies.

From an **evolutionary standpoint**, competition was essential for survival. Early human societies depended on strategic thinking, adaptability, and collective intelligence to secure resources and outmaneuver rival groups. These capabilities not only facilitated survival, but also laid the groundwork for the emergence of **cultural, intellectual, and economic systems**. Even in early development, individuals instinctively compare themselves with others, suggesting that competition is embedded in the cognitive architecture of human interaction.

What began as a mechanism for survival has evolved into a multifaceted force shaping ambition, identity, and societal progress.

In the digital age, this enduring instinct has migrated into **virtual environments**. The advent of **computers** and the rise of the **internet** have transformed how individuals interact, process information, and engage in competitive practices. Among these, video games have emerged as structured digital systems that encapsulate the competitive spirit, offering challenges against both algorithmic and human opponents.

These platforms replicate the essence of traditional competition by rewarding skill optimization, strategic decision-making, and the exploitation of system mechanics. As Murdock et al. [3] notes, individuals under competitive pressure are more likely to pursue

unconventional methods to gain an advantage, particularly when legitimate paths to success appear constrained. In online multiplayer gaming, elements such as **leaderboards**, **rankings**, and **public performance metrics** amplify this dynamic, motivating players to pursue victory through mastery, adaptation, or exploitation.

To fully understand how digital competition has taken root, it is essential to examine the historical trajectory that enabled it. The following sections trace the evolution of **computation**, the emergence of **networked systems**, and the rise of **video games** as influential cultural and competitive platforms. Through this historical lens, we observe how ancient instincts have been recontextualized into virtual domains where **skill**, **innovation**, and **adaptability** define success.

2 The Birth of Computation and the Internet: A New Digital Battleground

The foundations of digital competition are rooted in humanity's long-standing efforts to **automate calculation and reasoning**. From early counting instruments to mechanical systems, civilizations have persistently sought to expand their capacity for mathematical abstraction, an endeavor central to the advancement of commerce, science, and governance.

2.1 Early Computational Tools: From the Abacus to Mechanical Calculators

To improve accuracy and efficiency in arithmetic, ancient societies developed tools that facilitated numerical reasoning. The **abacus**, in use as early as 2400 BCE, was adopted across Mesopotamia, China, and Europe as a means of performing complex calculations with speed and consistency.

The 17th century marked a leap forward with the integration of mechanical principles into computation. One of the earliest examples was **Blaise Pascal's Adding Machine** (1642), a gear-based calculator capable of automating basic operations. This innovation reduced cognitive load and represented a pivotal moment in the shift toward mechanized problem-solving.

2.2 The 19th-Century Revolution: The Birth of Programmable Machines

The 19th century introduced a conceptual breakthrough: the idea of machines not limited to fixed operations, but capable of **programmable logic**. At the forefront of this transformation was **Charles Babbage**, whose design for the **Analytical Engine** anticipated core concepts of modern computing, such as conditional execution, memory storage, and input/output modules.

Building on Babbage's architecture, **Ada Lovelace** proposed that such a machine could manipulate not only numbers, but symbolic information. Her contributions, recognized as the first algorithm intended for execution by a machine, foreshadowed the rise of **general-purpose computing** and formalized the foundation for programming as a discipline.

2.3 The Acceleration of Computation in the 20th Century

Computational progress accelerated dramatically in the 20th century, spurred by global conflict and technological ambition. The move from mechanical to electronic systems redefined computation as a scalable, strategic resource.

2.3.1 World War II and the Role of Computation

The demands of **World War II** catalyzed a new era of computational innovation. **Alan Turing** formalized the concept of the **Turing Machine**, offering a theoretical model of algorithmic processing. At Bletchley Park, his work in cryptanalysis, particularly breaking the **Enigma cipher**, highlighted the wartime value of computational intelligence.

Parallel advancements produced operational systems like **Colossus** (UK, 1943) and **ENIAC** (US, 1945), capable of executing high-speed calculations for cryptography and ballistics. These achievements underscored the transformative potential of computation in military and scientific contexts.

2.3.2 Post-War Expansion: From Military to Commercial Computing

In the post-war era, computing spread from defense to academia and industry. The invention of the **transistor** (1947) and subsequent development of **integrated circuits** made computing more accessible and energy-efficient.

By the 1970s, companies like **IBM**, **Apple**, and **Microsoft** played central roles in the dissemination of **personal computing**, converting an elite technology into a ubiquitous tool for productivity, education, and leisure.

2.4 The Birth of the Internet: From Military Networks to Global Infrastructure

The next major evolution came with the interconnection of computers. The U.S. Department of Defense's **ARPANET** (late 1960s) pioneered **packet-switched networks** and **decentralized architectures**, making communication more efficient and resilient.

The introduction of **TCP/IP protocols** (1980s) and the **Domain Name System (DNS)** (1984) enabled standardized and human-friendly addressing. In 1991, **Tim Berners-Lee** launched the **World Wide Web**, layering hypertext on top of existing infrastructure and initiating a global shift in information access and interaction.

2.5 The Digital Arena: From Computing to Competition

With computational power democratized and global networks in place, digital spaces became arenas of performance and competition. Early LAN parties and cybercafés fostered community-driven play, where local matches evolved into organized events.



Figure 2.1: LAN party in the late 1990s, a grassroots precursor to modern esports.

Games like **Quake**, **Counter-Strike**, and **StarCraft** introduced mechanics that combined dexterity, strategy, and persistence, supported by matchmaking systems and persistent rankings. The shift from casual entertainment to **performance-based progression** gave rise to structured digital competition.

2.6 From Networks to Virtual Battlefields

The fusion of technology, connectivity, and community transformed games into **virtual battlegrounds** defined by measurable performance and equal opportunity. Online multiplayer systems enabled real-time global competition, while the surrounding infrastructure, streaming platforms, sponsorships, and professional leagues solidified gaming as a legitimate domain of public achievement.

This evolution culminated in the rise of **esports**, with professional players, teams, and audiences spanning the globe. What began as leisure evolved into a career path and cultural institution.

Today, the ancient drive to compete finds expression in these digital environments, where **technical skill**, **cognitive agility**, and **strategic depth** determine success.

3. THE EVOLUTION OF CHEATING: FROM DEBUGGING TOOLS TO AN ORGANIZED INDUSTRY

The history of cheating in video games is deeply intertwined with the evolution of the industry itself. What began as debugging tools for developers gradually transformed into a clandestine culture of modifications, and with the advent of online gaming, an organized and profitable industry with ties to cybercrime [4]. At the same time, the fight against cheating led to the development of **anti-cheat systems**, which evolved from simple integrity checks to sophisticated solutions incorporating artificial intelligence and deep system access [5].

1 The Early Days: Debugging Tools and Experimental Modifications

Before video games became a mainstream form of entertainment, early cheating emerged as a necessity within software development. In the 1970s and 1980s, developers embedded debugging tools within their games to facilitate testing. These systems allowed programmers to modify internal values such as lives, ammunition, and speed, enabling them to test different scenarios without playing through the entire game [6].

Some of these tools remained accessible in commercial releases, either unintentionally or as a deliberate choice by developers. This led to the creation of the first **cheat codes**, which were key combinations or commands that altered gameplay. One of the most iconic examples is the **Konami Code** (↑ ↑ ↓ ↓ ← → ← → B A), introduced in *Gradius* (1986) and later popularized in *Contra* (1988) [7]. While these codes were not designed to affect competition, they set a precedent for the intentional modification of gameplay.

2 The Rise of Cheating in Consoles and PC

As the video game industry grew throughout the 1980s and 1990s, cheating expanded beyond debugging tools and built-in cheat codes. With the advent of home consoles, external devices specifically designed to modify gameplay began to appear.

2.1 Cheat Devices: Game Genie, GameShark, and Pro Action Replay

The first dedicated cheat devices were **modification cartridges**, such as the **Game Genie** (1990) and the **GameShark** (1995). These hardware add-ons allowed players to inject custom modifications into a game's memory, altering key parameters such as health, ammunition, and physics [8]. While initially marketed as harmless tools for enhancing single-player experiences, they introduced the concept of **real-time game manipulation**, which would later be adapted for online environments.

On the PC side, the introduction of **memory editors** provided users with a more flexible and powerful way to modify game values at runtime. Early programs like **Cheat Engine** (2000) enabled players to search for specific in-game variables and alter them directly, bypassing the limitations of built-in cheat codes [9]. This shift from **developer-sanctioned cheats** to **unauthorized external modifications** laid the foundation for more advanced game-hacking techniques.

As gaming technology evolved, so did the tools available for modifying game memory structures. More sophisticated programs such as **ReClass.NET** emerged, allowing users to analyze and modify game memory at a structural level. Unlike traditional memory scanners, ReClass.NET enables users to reverse-engineer game memory layouts by dynamically inspecting class structures and data offsets in real time [10]. This advancement significantly improved the ability of cheat developers to create complex modifications, ranging from automatic aimbots to more intricate exploits that manipulate how game logic processes data.



Figure 3.1: Game Genie

These tools, originally designed for educational and debugging purposes, soon became widely used in cheat development, particularly in online gaming environments. As their complexity increased, anti-cheat systems had to adapt rapidly, resulting in an ongoing technological competition between cheat developers and security teams.

3 The Transition to Online Gaming: Cheating Becomes a Competitive Threat

The rise of online multiplayer gaming in the late 1990s and early 2000s fundamentally changed the impact of cheating. Unlike single-player cheats, which only affected the individual experience, online cheating directly disrupted fair competition [5]. As games like **Quake** (1996), **Counter-Strike** (1999), and **Diablo II** (2000) introduced matchmaking and ranking systems, the incentive to cheat increased dramatically [11].

As competitive integrity gained prominence, developers began to implement more aggressive measures against cheating, resulting in the creation of server-side and client-side anti-cheat mechanisms. However, as anti-cheat solutions became more sophisticated, cheating techniques also advanced, producing a continuous cycle of adaptation between game security teams and cheat developers.

This progression led to the development of the first organized cheat communities, where players and programmers shared exploits, scripts, and modified game clients. It also marked the beginning of a persistent competition between cheat developers and game security teams, as new forms of cheating emerged alongside increasingly complex anti-cheat measures.

4 The Birth and Evolution of Anti-Cheat Systems

As online gaming became increasingly competitive, the integrity of fair play was challenged by the widespread use of cheats. Unlike single-player modifications, which had no broader consequences, cheating in multiplayer environments compromised **matchmaking, rankings, and esports competitions**, creating frustration among players and damaging the credibility of competitive gaming.

Early attempts to mitigate cheating relied on **server-side validation**, where game servers monitored player actions for anomalies that suggested external manipulation. These checks aimed to detect unnatural behavior, such as **impossible movement speeds, improbable accuracy, and modifications to game assets** that altered visibility constraints. However, this approach had significant limitations, as many cheats operated entirely on the **client side**, modifying game data before it was transmitted to the server. As a result, more sophisticated **client-side anti-cheat solutions** were developed to monitor and detect unauthorized modifications directly within the player's system.

One of the first dedicated anti-cheat programs was **PunkBuster**, developed by Even Balance, Inc. and introduced in **2000**. Initially implemented in *Return to Castle Wolfenstein* and later in *Battlefield 1942*, PunkBuster marked a shift from basic server-side detection to **active client-side monitoring**. It worked by scanning a player's system for **known cheat signatures**, verifying **game file integrity**, and capturing periodic **in-game screenshots** to identify visual modifications such as **wallhacks**. Additionally, PunkBuster maintained a **global ban list**, preventing known cheaters from accessing protected servers. Despite these advancements, **cheat developers quickly adapted** by employing techniques such as **code obfuscation**, allowing their software to remain undetected. PunkBuster also faced criticism for its **aggressive scanning approach**, which occasionally resulted in **false positives**, banning legitimate players due to conflicts with background applications.

In **2002**, Valve introduced **Valve Anti-Cheat (VAC)** alongside *Counter-Strike 1.6*, implementing a more **automated and data-driven approach** to cheat detection. Unlike PunkBuster, which **immediately expelled** suspected cheaters, VAC introduced a **delayed banning system**. This strategy allowed suspected cheaters to continue playing while logging their infractions, making it harder for cheat developers to **identify detection triggers** and modify their cheats accordingly. In addition to **signature-based detection**, VAC utilized **heuristic analysis** to recognize **abnormal behavioral patterns**, refining its detection algorithms over time. By integrating **server-side validation** with **machine learning techniques**, VAC continuously improved its ability to detect and counteract evolving cheats. However, since it **operated at the user level** rather than the **kernel level**, advanced cheats that manipulated system memory with higher privileges could still **bypass its protections**.

As cheat developers refined their evasion techniques, more aggressive anti-cheat solutions emerged. One of the most notable advancements was **BattleEye**, introduced in 2004 and later adopted in games such as *Arma 2*, *Rainbow Six Siege*, and *PUBG*. Unlike VAC, which prioritized passive detection, BattleEye enforced stricter security measures by running with elevated system privileges. By operating at the kernel level, it could detect and block cheat software that manipulated game memory at a deeper level, making it significantly harder for external programs to alter in-game data without detection. This approach enabled BattleEye to intercept and prevent unauthorized modifications before they could affect gameplay, providing a more proactive defense against sophisticated cheats.

As competitive gaming and esports continued to grow, Riot Games introduced a new anti-cheat solution, Vanguard, designed specifically for *Valorant*. Released in 2020, Vanguard elevated anti-cheat enforcement by implementing persistent kernel-level monitoring. This system actively runs as a background process from the moment the operating system starts, enabling it to detect and prevent unauthorized modifications at the lowest levels of system memory. While this approach is one of the most invasive and effective anti-cheat solutions, its intrusive nature sparked controversy among players concerned

about privacy, as the software maintains continuous access to system processes even when the game is not running.

The evolution of anti-cheat systems reflects a continuous technological competition between cheat developers and game security teams. While early efforts relied on rudimentary server-side validation, modern solutions integrate behavioral analysis, machine learning, and kernel-level access to provide more robust protection. Despite these advancements, cheat developers continually adapt, ensuring that no anti-cheat system remains impenetrable for long. This dynamic perpetuates an ongoing cycle of exploitation and countermeasures in the digital landscape of competitive gaming.

This historical evolution has given rise to a wide variety of anti-cheat implementations, each shaped by the needs and challenges of its era.

The continuous development of anti-cheat solutions over the past two decades can be summarized through a timeline of key technologies. Figure 3.2 provides a chronological overview of the most influential anti-cheat systems, from early client-side detectors to modern kernel-level and behavior-based solutions.

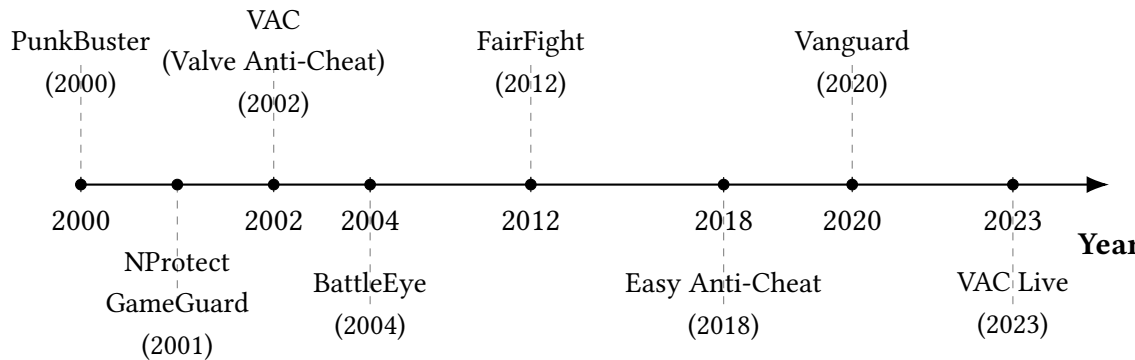


Figure 3.2: Chronological overview of key anti-cheat systems introduced between 2000 and 2023.

Among the systems depicted in the timeline, several stand out for their unique approach and impact.

nProtect GameGuard, launched in 2001, was among the first anti-cheat solutions to employ rootkit-like behavior. Widely used in Asian MMORPGs, it integrates deeply into the operating system, operating at the kernel level (ring 0), to prevent cheats from interacting with game processes. This deep integration has raised concerns about privacy and system stability [12].

FairFight, introduced in 2012, departed from traditional detection mechanisms by using server-side statistical analysis to flag abnormal player behavior. Instead of inspecting memory or processes, it relied on gameplay metrics such as accuracy, reaction time, and movement patterns. This approach made it less intrusive, but also vulnerable to both false positives and high-skill players [13].

Easy Anti-Cheat (EAC), which gained traction in 2018, became a popular choice for fast-paced competitive games. It combines kernel-level access with performance-friendly design and supports real-time detection of unauthorized drivers, injected DLLs, and tampering attempts, balancing robustness and user transparency [14].

Finally, VAC Live, unveiled in 2023, represents Valve's shift toward real-time, data-driven cheat prevention. Unlike the original VAC system, which relied on delayed bans, VAC Live is designed to detect and react to cheats during gameplay. This makes it a critical component in safeguarding the integrity of high-stakes esports matches [15].

5 The Commercialization of Cheating

The commercialization of video game cheating has evolved into a multifaceted ecosystem encompassing knowledge-sharing communities, grey-market resellers, and professionalized cheat providers. This transformation reflects a shift from informal exchanges to structured operations with varying degrees of legality and ethical considerations.

Platforms such as **UnKnoWnCheaTs** serve as non-commercial forums dedicated to the dissemination of information related to game hacking. These communities focus on educational content, including reverse engineering tutorials, memory manipulation techniques, and the release of free cheats contributed by community members. Notably, UnKnoWnCheaTs maintains a strict policy against the sale of cheats, emphasizing its commitment to a non-profit model centered on knowledge exchange [16].

In contrast, forums like **ElitePVPers** and **MPGH** operate as marketplaces where individuals can buy and sell cheats. These platforms facilitate the distribution of paid cheats through various models, including reselling arrangements and subscription-based access. Users often engage in discussions about becoming resellers, highlighting the existence of structured networks for cheat distribution [17].

Beyond community forums, a segment of the cheating industry comprises professionalized providers offering sophisticated cheat solutions. These entities develop and sell advanced cheating tools, often featuring capabilities such as kernel-level access, hardware ID spoofing, and real-time updates to circumvent anti-cheat measures. The following table summarizes prominent cheat providers active in the CS2 ecosystem:

Provider	Released	Subscription Price	Access Model
Neverlose [18]	2020	€29/month, €69/3 months, €129/year	Invite-only
Memesense [19]	2023	\$4/14 days, \$8/month, \$12/3 months	Public access
Fatality.win [20]	2022	\$33/month	Invite-only
Project Infinity [21]	2017	€14.99/month, €29.99/3 months, €99.99 lifetime	Public access
Midnight [22]	2019	£5.15/month	Public access
Onetap [23]	2024	\$20/month	Public access
Skeet (Game-sense) [24]	2016	\$20/month	Invite-only + HWID locked

Table 3.1: Overview of major CS2 cheat providers, including release year, pricing models, and access restrictions.

From Historical Context to Market Impact

Having explored the historical and sociotechnical evolution of video game cheating, from its inception with debugging tools to its current status as a sophisticated and commercialized industry, we now shift our focus to contemporary market dynamics and their ramifications on gaming communities.

The subsequent chapters will examine the economic structures underpinning the cheating industry, analyzing how monetized cheating services influence player behavior, community trust, and the overall integrity of online gaming environments. Through this analysis, we aim to elucidate the broader social and economic impacts of cheating within the gaming ecosystem.

With the historical framework established, we now proceed to assess the market-driven aspects that perpetuate and shape the landscape of video game cheating.

4. THE PREVALENCE AND IMPACT OF CHEATING IN ONLINE MULTIPLAYER GAMES

Building upon the historical and sociotechnical evolution of video game cheating, from its origins in debugging tools to its current manifestation as a sophisticated, commercialized industry, we now examine its pervasive impact on contemporary gaming communities and market dynamics.

Cheating in online multiplayer games constitutes an escalating systemic threat that undermines fairness, erodes player trust, and destabilizes gaming ecosystems. This chapter delves into the multifaceted consequences of cheating, with particular emphasis on the evolving landscape surrounding *Counter-Strike 2* (CS2), and argues that addressing this issue demands an integrated response encompassing technical, legal, and social countermeasures.

As competitive online titles continue to thrive, especially those featuring ranked ladders, esports ecosystems, and in-game economies, the incentives to obtain unfair advantages intensify. This dynamic perpetuates a relentless arms race between cheat developers and game studios, necessitating not only robust technological defenses but also a comprehensive understanding of the socio-economic impacts of cheating.

1 Cheating as a Critical Threat to Player Retention

Cheating is not merely a minor inconvenience; it stands as a significant factor contributing to player attrition and dissatisfaction. A global survey conducted by Irdeto revealed that 60% of players reported a negative impact on their gaming experience due to cheating, and 77% indicated they would discontinue playing a game if cheating became widespread [25].

Table 4.1: Survey Results on Cheating in Online Games (Irdeto, 2018)

Survey Statement	Percentage of Respondents
Experienced negative impact due to cheating	60%
Would stop playing a game because of cheaters	77%
Support permanent bans for cheaters	69%
Believe developers are not doing enough	55%

These findings underscore that cheating not only generates immediate frustration but also carries long-term economic implications. Since player engagement drives monetization through mechanisms such as battle passes, cosmetic sales, and advertising, a decline in player retention fueled by cheating poses a direct threat to publishers’ revenue streams.

Ultimately, when players perceive ranked systems, matchmaking, or competitive outcomes as compromised, they are likely to migrate toward alternative titles, thereby undermining the sustainability and vitality of the gaming community.

2 Case Study: Cheating in *Counter-Strike 2*

Counter-Strike 2 (CS2) stands as a prominent example of modern cheating dynamics within competitive first-person shooter games. Its expansive player base and vibrant esports scene make it both an attractive target for cheat developers and a critical testing ground for anti-cheat innovations.

To contextualize the scale of CS2’s ecosystem, Figure 4.1 illustrates the evolution of its player base from January 2024 to April 2025, based on data from SteamCharts:

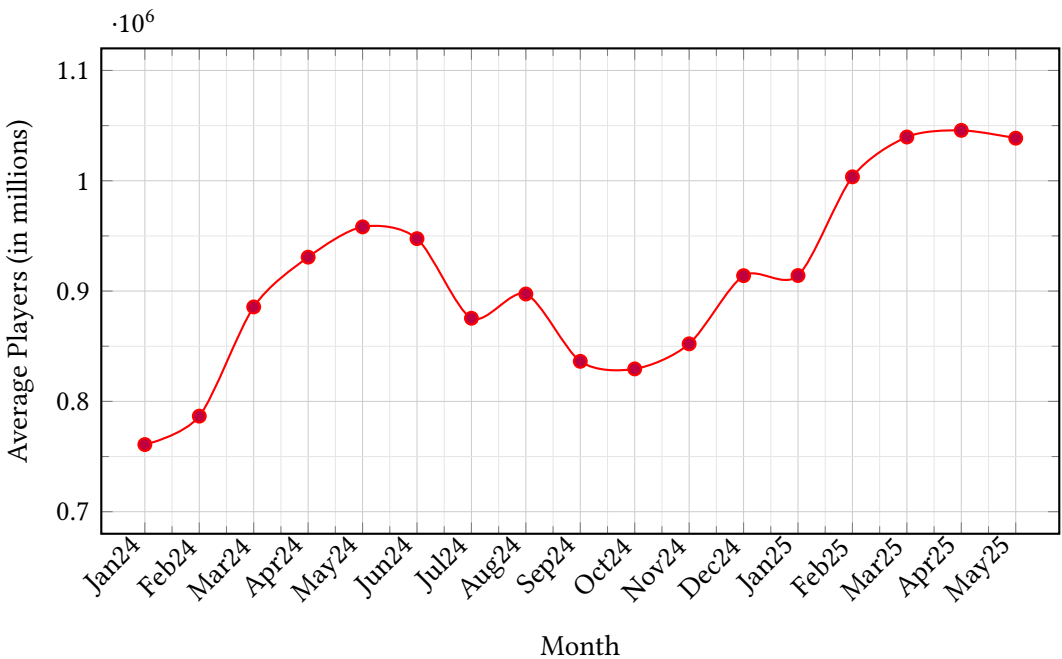


Figure 4.1: Evolution of Average CS2 Player Counts (January 2024 to May 2025). Source: SteamCharts.

The consistent growth in CS2’s player base underscores the game’s enduring popularity and its pivotal role in the competitive FPS landscape. However, this expansion introduces increased complexity; a larger and more diverse player community amplifies the challenges associated with maintaining competitive integrity.

Data from recent enforcement activities further highlights the scale and persistence of the issue [26]:

Table 4.2: Activity of Banned Accounts Prior to Enforcement (January to April 2024)

Metric	Estimated Value
Average delay between last match and VAC ban	20 days
Percentage of bans issued within 24 hours	28%
Total matches played by banned accounts	Over 32,000
Percentage of matches with only one banned player	97%
Evidence of case farming behavior	Not significant

These observations reveal that cheaters are not isolated anomalies; rather, they are embedded within regular matchmaking sessions, further complicating detection and enforcement efforts.

2.1 Trust Factor and Reporting System Limitations

To mitigate cheating and enhance matchmaking quality, Valve implemented the *Trust Factor* system in *Counter-Strike 2* (CS2).

Definition: Trust Factor Explained

The Trust Factor system aggregates multiple indicators such as VAC bans, player reports, account age, and Steam community standing to adjust matchmaking quality. Its opacity aims to prevent manipulation but can lead to confusion and potential exploitation.

While the Trust Factor system provides a foundation for fair matchmaking, its lack of transparency presents two significant challenges: (1) players have limited means to understand or improve their Trust Factor; and (2) third-party services offering *report bots* and *commend bots* can manipulate trust metrics for profit [27].

2.2 Economic and Psychological Impact

The ramifications of cheating extend beyond gameplay, inflicting substantial economic and psychological harm. In early 2025, a significant VAC enforcement wave resulted in the permanent banning of accounts with inventories collectively valued at approximately \$2 million [28]. Although this action was lauded as a crackdown on cheaters, it also raised concerns about transparency and the potential for false positives.

Definition: VAC Ban

A Valve Anti-Cheat (VAC) ban is a permanent restriction imposed when cheating activity is detected. It disables multiplayer functionality and trading privileges associated with the affected title.

For players who have invested considerable time, resources, and personal attachment into their accounts, the psychological toll of encountering cheaters or being wrongfully penalized can be profound, leading to disengagement and a decline in community loyalty.

3 Community Perception and the Culture of Cynicism

Cheating incidents are highly visible across social platforms such as Reddit, YouTube, and Twitch. High-profile streamers encountering cheaters during broadcasts exacerbate perceptions of widespread dishonesty, even when empirical cheating rates remain relatively low.

This feedback loop fosters a culture of cynicism, where player trust in matchmaking systems and anti-cheat technologies diminishes, ultimately endangering long-term community stability.

4 Legal and Ethical Responses to Cheat Development

Recognizing the limitations of technical interventions, game developers have increasingly turned to legal avenues to combat cheat creators. A notable example is Bungie's legal action against Mihai Claudiu-Florentin, the operator of VeteranCheats, which resulted in a default judgment awarding Bungie **\$12,059,912.98** in damages. This amount encompassed statutory damages under the Digital Millennium Copyright Act (DMCA), actual damages for copyright infringement, and attorneys' fees and costs [29].

This legal precedent has encouraged other companies, such as Riot Games, Ubisoft, and Activision, to adopt similar strategies in protecting their intellectual property and maintaining fair play within their gaming communities.

5 Anti-Cheat Technologies: Current Trends and Limitations

Despite significant advancements, contemporary anti-cheat technologies predominantly operate in a reactive capacity, often lagging behind the evolving tactics of cheat developers.

Valve's *Valve Anti-Cheat* (VAC) system primarily employs signature-based detection methods, scanning for known cheat signatures within the game's memory. This approach renders VAC susceptible to evasion techniques such as code obfuscation and manual mapping, which can bypass signature recognition mechanisms [30].

Conversely, Riot Games' *Vanguard* system adopts a more intrusive strategy by operating at the kernel level, granting it deep access to system processes. While this enhances its capability to detect sophisticated cheats, it has raised substantial privacy and security concerns among users due to its persistent presence and high-level system access [31].

Emerging methodologies, including AI-driven behavioral analysis and the utilization of secure enclave technologies, present promising avenues for proactive cheat detection. AI-based systems can analyze player behavior in real-time to identify anomalies indicative of cheating, while secure enclaves offer isolated environments that protect sensitive computations from tampering. However, these approaches introduce new challenges, such as the potential for false positives and concerns regarding user data privacy [32, 33].

The ongoing battle between cheat developers and anti-cheat engineers constitutes a dynamic and relentless arms race, necessitating continuous innovation and adaptation to preserve the integrity of competitive gaming environments.

Key Takeaways:

- Cheating poses a direct threat to player retention, game reputation, and revenue streams.
- Systems like *Trust Factor* require increased transparency and user engagement to be effective.
- Legal actions are a critical component in combating organized cheat development and distribution.
- The fight against cheating is an ongoing, adaptive arms race necessitating continual technological and strategic innovation.

5. INJECTION AND DETECTION TECHNIQUES

From Anti-Cheat Strategies to Injection Mechanisms

In the preceding chapters, we examined the multifaceted approaches employed to combat cheating in online multiplayer games, encompassing technical, legal, and social strategies. Despite these efforts, cheat developers continually adapt, employing sophisticated methods to circumvent detection. A prevalent technique involves the injection of dynamic-link libraries (DLLs) into the game's process space, enabling unauthorized manipulation of game behavior.

This chapter delves into the technical intricacies of such injection methods, elucidating how they function and the challenges they pose to anti-cheat systems. By understanding these mechanisms, we can better appreciate the ongoing cat-and-mouse dynamic between cheat developers and game security measures.

1 The Role of DLL Injection in Cheat Development

In the realm of professional game cheating, particularly within competitive titles like *Counter-Strike 2*, the injection of dynamic-link libraries (DLLs) into the game's process is a prevalent technique. This method allows unauthorized code to execute within the game's context, facilitating various forms of cheating.

Definition: Dynamic-Link Library (DLL)

A *Dynamic-Link Library (DLL)* is a module containing functions and data that can be used by multiple programs simultaneously. In the context of game cheating, DLLs are injected into a game's process to modify its behavior at runtime, enabling functionalities not intended by the original developers [34].

Injecting a DLL permits the cheat to operate concurrently with the game, granting the ability to:

- **Hook game functions:** Altering or extending the behavior of existing game functions.
- **Modify memory:** Changing in-game variables such as player speed, ammunition count, or recoil patterns.

- **Read internal data:** Accessing sensitive information like player positions or health status.
- **Render overlays:** Displaying additional visuals (e.g., radar, ESP) directly onto the game screen.
- **Automate actions:** Executing tasks like aiming or firing without user input.

Due to the deep integration achieved through DLL injection, anti-cheat systems like **Valve Anti-Cheat (VAC)** prioritize detecting and preventing such unauthorized modifications to maintain game integrity.

2 Overview of Code Injection Techniques

Building upon the foundational understanding of DLL injection, it is imperative to explore the broader spectrum of code injection methodologies employed in cheat development. **Code injection** refers to the practice of introducing unauthorized code into a running process, thereby altering its behavior. In the context of game cheating, this often involves inserting a DLL or custom code into the game's memory space.

Definition: Code Injection

Code injection is a technique whereby an attacker introduces malicious code into a vulnerable application, causing the application to execute unintended commands. This is typically achieved by exploiting inadequate input validation, allowing the attacker to manipulate the program's execution flow and potentially gain unauthorized access or control over the system [35].

Traditional Methods

Historically, cheat developers have leveraged standard Windows APIs to facilitate code injection. Two of the most prevalent functions are:

- **LoadLibrary:** Loads a specified DLL into the address space of the calling process.
- **CreateRemoteThread:** Creates a thread that runs in the virtual address space of another process.

The conventional approach entails allocating memory within the target process, writing the path of the malicious DLL into this memory, and then invoking **CreateRemoteThread** to execute **LoadLibrary**, thereby loading the DLL into the target process [36].

Definition: Classic DLL Injection Workflow

1. Obtain a handle to the target process using `OpenProcess`.
2. Allocate memory in the target process with `VirtualAllocEx`.
3. Write the DLL path into the allocated memory via `WriteProcessMemory`.
4. Retrieve the address of `LoadLibrary` using `GetProcAddress`.
5. Create a remote thread in the target process to execute `LoadLibrary`, loading the DLL.

While effective in earlier implementations, these methods have become increasingly detectable by modern anti-cheat systems due to their reliance on well-known API calls and the presence of the malicious DLL on disk. Consequently, cheat developers have evolved their techniques to employ more sophisticated and stealthy injection methods, which will be discussed in subsequent sections.

Manual Mapping: A Stealthier Injection Strategy

As anti-cheat systems became more adept at detecting conventional injection methods, cheat developers responded with more sophisticated strategies. One such method is known as **Manual Mapping**, which remains a favored approach due to its stealth and flexibility.

Definition: Manual Mapping

Manual Mapping is a technique in which a DLL is manually loaded into the memory space of a target process without invoking standard Windows loader functions like `LoadLibrary`. This bypasses many traditional detection mechanisms, as the injected module does not appear in the process's loaded module list and avoids creating suspicious threads [37, 38].

Unlike traditional injection, Manual Mapping involves manually parsing the DLL's headers, resolving imports, handling relocations, and executing the entry point, steps that are normally handled by the operating system. This provides the attacker with full control over how and when the code is executed.

In particular, manual mapping enables:

- **Avoidance of API hooks:** Since standard APIs are not called, API-based anti-cheat hooks are ineffective.
- **Concealed presence:** The DLL does not appear in module snapshots or standard tool output.
- **Flexible entry execution:** Control over when and how the DLL's code is run (e.g., hijacking existing threads).

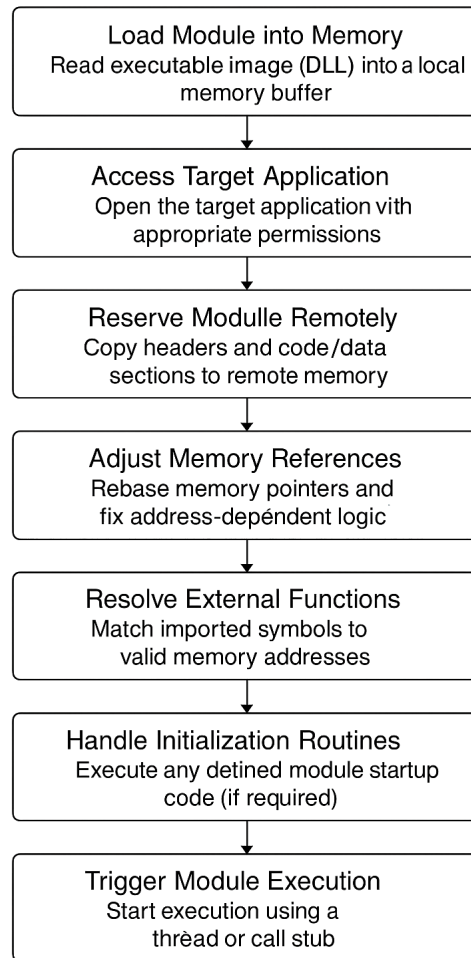


Figure 5.1: Simplified flowchart of dynamic module injection using Manual Mapping.

As part of this research, we developed a custom injector implementing manual mapping, as outlined in Figure 5.1. The source code is available in the injector branch of the accompanying repository.

3 Detection Techniques in Anti-Cheat Systems

Building upon the previously discussed injection methodologies, it is imperative to understand the countermeasures employed by modern anti-cheat systems to detect and mitigate unauthorized code execution. The techniques delineated in this section are not merely theoretical constructs; they are derived from meticulous reverse engineering of modules extracted directly from the game's memory during runtime. This analysis provides concrete evidence supporting the mechanisms and strategies outlined below.

Anti-cheat systems, such as Valve Anti-Cheat (VAC), deploy a multifaceted array of detection methods aimed at identifying anomalies indicative of cheating. These methods

focus on uncovering abnormal memory modifications, unauthorized module injections, and manipulations of trusted system structures.

Thread Local Storage (TLS) Callbacks

An initial opportunity for anti-cheat systems to detect injected code arises during the program's startup phase. Windows operating systems support the execution of special functions known as **Thread Local Storage (TLS) callbacks**, which are invoked automatically when a thread is created.

Definition: Thread Local Storage (TLS) Callback

A *TLS callback* is a function that the Windows operating system automatically calls when a thread is created or terminated. These callbacks are typically used for initializing or cleaning up thread-specific data. However, they can also be leveraged by anti-cheat systems to perform early memory inspections before the main application code executes [39].

Anti-cheat mechanisms utilize this early execution stage to scan memory and inspect the runtime environment, often before the main game code begins. If an injected DLL includes TLS callbacks that are not properly removed or disabled, the cheat risks immediate detection during process initialization.

Start Address Integrity Checks

Beyond startup checks, anti-cheat systems continuously monitor thread creation during gameplay. A common detection method involves verifying the **start address** of each thread.

Definition: Thread Start Address

The *start address* is the memory location where a thread begins its execution. Legitimate threads typically start within official game modules, such as `client.dll` or other trusted libraries. Threads initiating from addresses outside these known modules are considered suspicious and may indicate injected code or shellcode execution [40].

This method is particularly effective at detecting poorly concealed code injections or unauthorized shellcode executions.

Virtual Method Table (VMT) Hook Detection

Another detection strategy focuses on the integrity of in-game objects. In object-oriented engines, game entities often utilize **Virtual Method Tables (VMTs)** to organize their functions.

Definition: Virtual Method Table (VMT)

A *Virtual Method Table (VMT)* is a structure used in object-oriented programming to map object methods to their actual function addresses. By modifying a VMT, an attacker can alter how a game object behaves, enabling the hijacking of functions such as rendering or input handling [41].

Cheats often modify these tables to hijack behaviors. Anti-cheat systems can identify tampered VMTs by verifying:

- Whether the VMT points to unexpected memory regions.
- Whether the methods referenced fall outside legitimate game or system modules.

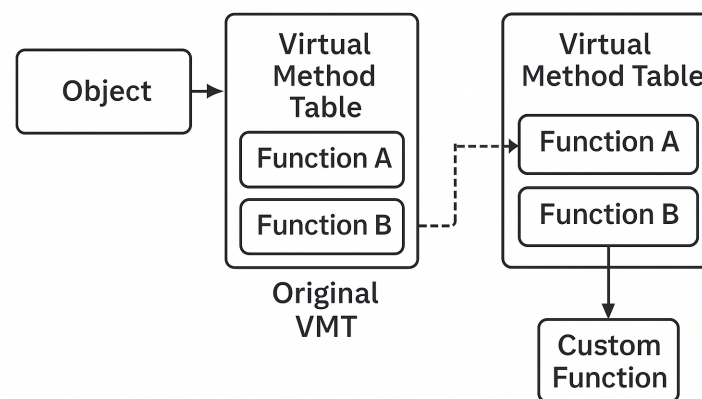


Figure 5.2: Example of VMT Hooking.

Due to their clear memory footprint, traditional VMT hooks are now considered relatively high-risk in modern cheat development.

4 Extended VAC Capabilities and Developer Countermeasures

Building upon the previously discussed detection methodologies, **Valve Anti-Cheat (VAC)** has evolved to incorporate a comprehensive suite of system-level monitoring techniques aimed at identifying and mitigating unauthorized code execution. These advancements extend beyond basic memory and thread inspections, encompassing a broader surveillance of the operating environment.

Advanced Detection Mechanisms

VAC's enhanced detection strategies include:

- **File System Scanning:** VAC actively scans the file system for known cheat artifacts, such as specific file names, hashes, or signatures associated with cheating tools [42].
- **Process and Memory Inspection:** It inspects running processes for suspicious memory regions or injected modules that deviate from expected patterns [42].
- **Registry Monitoring:** VAC monitors the Windows registry for abnormal entries that may indicate the presence of persistent malware or cheat configurations [42].
- **Inter-Process Communication Tracking:** It tracks inter-process activity to detect unauthorized access or manipulation of the game process by external applications [42].

These multifaceted approaches enable VAC to construct a holistic view of system behavior, thereby enhancing its ability to detect sophisticated cheating mechanisms that may not be evident through isolated memory analysis.

Developer Countermeasures

In response to VAC's advanced detection capabilities, cheat developers have devised a range of countermeasures aimed at evading detection:

- **String Encryption:** Obfuscating identifiable patterns in memory by encrypting strings, making it more challenging for signature-based detection systems to recognize cheat-related code [43].
- **Name Randomization:** Dynamically changing module and window identifiers to avoid blacklisting and hinder heuristic analysis.
- **Polymorphic Code:** Employing self-modifying code that alters its structure upon each execution while preserving its functionality, thereby evading pattern recognition [44].
- **Fileless Execution:** Operating entirely from memory without writing to disk, thus minimizing the forensic footprint and circumventing file-based detection mechanisms [45].

These sophisticated evasion techniques exemplify the ongoing arms race between cheat developers and anti-cheat systems. As VAC continues to refine its detection methodologies, cheat developers correspondingly adapt their strategies to maintain operational stealth.

The Ongoing Arms Race

The dynamic interplay between anti-cheat technologies and cheat development fosters a continuous cycle of innovation on both fronts. VAC's commitment to enhancing its detection capabilities is met with equally determined efforts by cheat developers to devise novel evasion tactics. This perpetual contest underscores the necessity for ongoing research and development in anti-cheat technologies to preserve the integrity of competitive gaming environments.

6. HOOKING AND REVERSING: CORE TECHNIQUES OF OUR CHEAT FRAMEWORK

Following the successful injection of our module and the circumvention of anti-cheat defenses, two tightly integrated techniques become central to the operation of our cheat: **reverse engineering** and **function hooking**. These mechanisms allow for in-depth manipulation of game internals, enabling the cheat to operate with both precision and stealth.

Reverse engineering provides the necessary insight into undocumented functions, memory layouts, and internal behaviors of the game engine. Function hooking, in turn, leverages this knowledge to intercept key routines, modify logic at runtime, and inject custom functionalities without breaking the expected control flow of the application.

Definition: Function Hooking

Function hooking is the practice of redirecting a function's execution flow to custom code. This technique enables interception, observation, or alteration of program behavior at runtime, without requiring access to the source code.

Among the various hooking techniques explored during our implementation, the **trampoline hook** proved to be the most robust and least intrusive. It allows us to reroute execution to our handler while preserving the original logic and avoiding stability issues that often arise from more aggressive methods.

This chapter provides a focused analysis of trampoline-based interception, its implementation via the MinHook library, and its vital role in achieving stable, persistent in-game modifications that remain hidden from both users and anti-cheat mechanisms.

1 Reverse Engineering and Function Discovery

Before any hooking can take place, one must first determine which functions are responsible for the in-game behaviors to be modified. This task requires a careful process of reverse engineering, analyzing the game's internal binaries, particularly `client.dll`, which encapsulates most of the client-side logic.

Using a combination of static analysis tools such as *IDA Pro* and *Ghidra*, along with dynamic debuggers like *x64dbg*, we disassembled code, examined symbols, and traced runtime execution to identify candidate functions for interception.

To bypass Address Space Layout Randomization (ASLR), we extracted **function signatures**, short, distinctive byte patterns unique to each function. Our injected DLL performs runtime pattern scanning to locate these signatures and apply hooks dynamically. This guarantees compatibility across executions and system reboots.

Unlike absolute address targeting, which fails under ASLR, our technique relies on scanning instruction bytes. Signatures are constructed with partial wildcards (??) to accommodate variable components like offsets or pointers, which may shift across builds.

For example, the following byte pattern is used to locate the view matrix computation within `client.dll`:

```
1 "48 8D ?? ?? ?? ?? ?? 48 C1 E0 06 48 03 C1 C3 CC CC"
```

Here, wildcard bytes increase pattern robustness by matching essential instruction sequences while tolerating relocation or minor binary differences. The full implementation of our pattern scanning logic resides in the `helper.cpp` module of our codebase.

2 Why Trampoline Hooks Are Effective

Trampoline hooks offer a highly effective mechanism for real-time manipulation of game logic while minimizing the footprint detectable by anti-cheat systems:

- They preserve control flow, allowing seamless continuation of the original function after the cheat logic executes.
- They maintain stack consistency and valid return addresses, reducing exposure to techniques like return address verification used by VAC.
- They avoid allocating and executing new memory pages, lowering the chances of detection by memory scanners.

This balance of flexibility, stealth, and resilience makes trampoline hooking the preferred method for runtime code interception in advanced cheat frameworks like ours.

3 Implementing Hooks with MinHook

To safely and efficiently intercept internal game functions, we rely on **MinHook**, a lightweight, open-source library tailored for x86 and x64 Windows systems. Following the function discovery and signature matching steps described earlier, MinHook enables us to install trampoline hooks with minimal code and maximal reliability.

Definition: MinHook

MinHook is a minimalistic C/C++ library that simplifies function hooking by handling instruction relocation, trampoline setup, and thread-safe patching. It is widely used in both research and production environments for its efficiency and robustness.

Our hook implementation follows a consistent structure:

- Define a typedef for the original function's signature.
- Declare a static pointer to store the trampoline (original) function.
- Implement a static method to contain the custom cheat logic.

Example: World Color Modulation Hook

```
1 // Class responsible for world color modulation hook
2 class WorldModulation {
3 public:
4     typedef void* (__fastcall* ModulateWorldColorFn)(...);
5     static ModulateWorldColorFn oModulateWorldColor;
6
7     static void* __fastcall hModulateWorldColor(...);
8 };
9
10 WorldModulation m_WorldModulation;
```

To install the hook using MinHook:

```
1 // Function to initialize and install the hook
2 void InitHook() {
3     MH_Initialize(); // Initialize MinHook
4
5     MH_CreateHook((LPVOID)TargetAddress, &WorldModulation::
6         hModulateWorldColor,
7         (LPVOID*)&WorldModulation::oModulateWorldColor);
8
9     MH_EnableHook((LPVOID)TargetAddress); // Activate the hook
10 }
```

The custom hook modifies behavior only if a specific global flag is active:

```
1 // Hooked function definition
2 void* __fastcall WorldModulation::hModulateWorldColor(...) {
3     if (!Globals::worldModulation)
4         return oModulateWorldColor(...); // Call original if disabled
5
6     for (int i = 0; i < pWorld->count; i++) {
7         auto& obj = pWorld->array[i];
8         obj.SetColor(Globals::worldModulationColor);
9     }
10
11     return oModulateWorldColor(...);
12 }
```

This selective execution strategy ensures that the cheat logic remains unobtrusive, maintaining performance and stability when inactive.

4 Final Thoughts on Trampoline Hooking

Trampoline hooking offers a powerful, stealthy, and modular mechanism for real-time manipulation of gameplay logic. By preserving control flow, maintaining valid return addresses, and avoiding allocation of suspicious memory regions, it reduces the likelihood of detection by modern anti-cheat systems.

The methodology presented, comprising targeted **reverse engineering**, dynamic signature scanning, and robust hook deployment via **MinHook**, constitutes the core of our cheat framework. This process has enabled us to identify and intercept internal game functions with precision, even across updates or memory relocations.

In the following chapters, we will present the foundations of the reverse engineering process that underpins our framework. This includes a detailed account of the discovered functions, their roles within the game's execution flow, the internal data structures used to manage state, and the architecture of the game engine responsible for organizing and updating game entities in real time.

7. INTERNAL GAME STRUCTURE: LOGIC BEHIND COUNTER-STRIKE 2

From Hooking to Context: Understanding Game Logic Internals

Following the development of our hooking infrastructure and the application of reverse engineering techniques, it becomes essential to explore the internal structures that underpin gameplay in *Counter-Strike 2*. While function interception provides a powerful interface for altering behavior at runtime, a deep understanding of the game's internal architecture is required to apply these modifications meaningfully and with precision.

This chapter introduces the logical foundation of the game engine, beginning with the entity system. By dissecting how entities are created, managed, and manipulated within memory, we establish the context needed to target specific gameplay mechanics, ranging from player visibility and health to movement, projectiles, and animations. Such knowledge is indispensable for implementing cheats that are not only effective, but stable and adaptable across game versions.

1 Understanding Entity Lists in Video Games

In modern game engine, particularly those designed for real-time, multiplayer environments, an **entity list** is a fundamental data structure responsible for managing all interactive objects within the game world. These entities include players, non-player characters (NPCs), projectiles, grenades, environmental props, and more.

Definition: Entity

An *entity* is any dynamic object in the game world, such as a player, NPC, projectile, or interactive prop, managed and updated by the engine each frame.

At the core, each entity is an instance of a class containing essential data such as position, health, team ID, state flags, and animation or input components. The engine maintains and updates these entities each frame to simulate gameplay logic, render visuals, process collisions, and evaluate state-based behavior such as death, visibility, or interaction.

1.1 Common Structures for Entity Lists

Entity lists are implemented using various structures, depending on the performance needs and memory model of the engine:

- **Array of Entity Objects:** A flat, contiguous layout offering fast access, but limited flexibility.
- **Array of Pointers:** A static array referencing dynamically allocated entities. Frequently used in older engine architectures.
- **std::vector:** A dynamic container ideal for runtime creation and destruction of entities.
- **Linked List:** A flexible memory structure where each node points to the next entity, at the cost of linear traversal time.
- **HashMap / Tree:** Enables fast lookup and categorization of entities based on IDs or class types.

Definition: Paged Entity System

A *paged entity system* is a memory management technique in which entities are grouped into fixed-size memory pages. This approach enhances scalability and performance, improves memory safety, and prevents access violations when entities are dynamically destroyed or reallocated.

While earlier iterations of the Source engine (such as CS:GO) utilized a simple flat array of entity pointers for entity management, the modern Source 2 engine in Counter-Strike 2 introduces a **paged entity system**. This design organizes entities into memory pages and utilizes safe handle-based referencing, offering greater scalability, safety, and efficiency across dynamically allocated objects.

2 Entity Organization in Counter-Strike 2

In *Counter-Strike 2*, every interactive object, whether a player, weapon, projectile, or environmental element, is internally represented as a dynamic **entity**. These entities are managed using unique *handles*, lightweight references that combine an entity index and a generation counter to ensure safe access and lifecycle management.

Definition: Handle

A *handle* is a compact reference to an entity, typically composed of an index and a generation counter. It enables efficient and safe resolution of entities, ensuring that references remain valid even as entities are destroyed and recreated, thus preventing access to stale or invalid memory.

This handle-based system enables robust referencing, safeguarding against invalid memory access even in highly dynamic environments where entities frequently appear and disappear during gameplay.

To map a handle to an actual entity in memory, the Source 2 engine employs a **two-level paging system**, a mechanism conceptually similar to virtual memory in modern operating systems. This approach balances memory efficiency, scalability, and safety, particularly as entity counts grow in large matches or complex game scenarios.

When analyzing player-specific data structures, two core classes emerge as critical pillars:

- **CBasePlayerController**, which manages high-level metadata about a player, including their sanitized name, Steam ID, current input state, and a handle linking to their in-world representation (the pawn).
- **C_CSPlayerPawn**, which represents the player's physical presence within the game world, storing vital gameplay attributes such as position, health, team affiliation, view angles, and animation states.

Both player-related classes inherit from **C_BaseEntity**, the core base class that provides common properties, behaviors, and networking support for all entities in the engine.

The following are simplified and annotated representations of these structures:

1. Base Entity – C_BaseEntity

```
1  /*****
2   * C_BaseEntity:
3   * Base class for all entities in Source 2.
4   *****/
5  class C_BaseEntity {
6  public:
7      CHandle<C_BaseEntity>    m_hEffectEntity;    // Effect entity
8      CHandle<C_BaseEntity>    m_hOwnerEntity;     // Owner
9      CHandle<C_BaseEntity>    m_hGroundEntity;    // Entity standing on
10     CBodyComponent::Storage_t m_CBodyComponent;  // Physics
11     int32_t                  m_iHealth;          // Entity health
12     Vector3                   m_vPosition;       // World-space
13                               position
14
15     // ... Additional fields (e.g., team, animation state)
16 };
```

Listing 7.1: C_BaseEntity – Core Class for All Entities

2. Player Controller – CBasePlayerController

```
1  /*****
2   * CBasePlayerController:
3   * Manages metadata, Steam ID, input, and pawn handle.
4   *****/
5  class CBasePlayerController : public C_BaseEntity {
6  public:
7      std::string          m_sSanitizedPlayerName; // Player name
8      uint64_t             m_steamID;              // Steam Id
9      InputState_t         m_inputState;           // Current input
10     CHandle<C_CSPlayerPawn> m_hPawn;              // Handle to pawn
11     uint32_t              m_iTeamNum;             // Team ID
12     bool                  m_bIsConnected;         // Connection
13     status
14
15     // ... Other metadata (e.g., score, latency)
16 };
```

Listing 7.2: CBasePlayerController – Player Metadata and Input Handler

3. Player Pawn – C_CSPlayerPawn

```
1  /*****
2   * C_CSPlayerPawn:
3   * Represents the physical player in the world.
4   *****/
5  class C_CSPlayerPawn : public C_BaseEntity {
6  public:
7      Vector3              m_vPosition;            // World position
8      uint32_t             m_iHealth;              // Health
9      uint32_t             m_iTeamNum;             // Team ID
10     QAngle                m_angViewAngles;        // View angles
11     AnimationState_t      m_AnimState;            // Animation state
12
13     // ... Other attributes
14 };
```

Listing 7.3: C_CSPlayerPawn – In-World Representation of the Player

This class hierarchy enables a clear separation of concerns between the *logical control* of the player (controller) and their *physical simulation* within the game world (pawn). This design also accommodates edge cases such as spectator modes, bot logic, and replay systems, where a controller may exist without an active pawn.

3 Entity List Traversal

Having established the structural foundations of player entities, the next step involves accessing and enumerating these entities during runtime. This process, known as **entity list traversal**, is essential for real-time game modification, data extraction, and the development of external tools.

3.1 Locating and Identifying the Entity List in Memory

In the absence of **debugging symbols** or **SDK access**, a common scenario in protected or closed-source games, discovering the location and structure of the **entity list** becomes a crucial part of the reverse engineering process. Two primary approaches are typically employed: *dynamic memory inspection* and *static binary analysis*.

Dynamic Approach: Real-Time Memory Inspection

Dynamic memory inspection focuses on observing changes in the game's memory while it is running, providing a practical way to locate the entity list without dissecting the entire executable.

The discovery process generally follows these steps:

1. **Scan for the Local Player's Position:** Search for changing float values that represent the player's coordinates (X, Y, Z) during movement.
2. **Filter Out Non-Relevant Addresses:** Discard addresses that stop updating when the game is paused or unfocused, as they typically reflect cached or rendering-only data.
3. **Analyze Accessing Instructions:** Use memory inspection tools to detect which instructions interact with these addresses.
4. **Detect Multi-Entity Patterns:** Identify if the same instruction accesses positions of multiple players, suggesting iteration over an entity container.
5. **Recover Entity Base Pointers:** Subtract known field offsets (e.g., `m_vPosition`) from accessed addresses to obtain base pointers to entities.
6. **Locate the Entity List Structure:** Search for contiguous memory regions holding these base pointers, checking for uniform spacing, characteristic of structured storage (arrays or paged layouts).
7. **Confirm and Generalize:** Build a traversal loop using pointer arithmetic to iterate through the list and access relevant entity attributes such as health, team ID, and state flags.

Important: Not all structures resembling entity lists are gameplay-relevant. Careful validation is necessary to avoid mistaking render-only data for true gameplay entities.

Static Approach: Reverse Engineering the Binary

When dynamic analysis alone is insufficient or incomplete, static binary analysis becomes essential. This method involves disassembling and inspecting the game executable to directly uncover the internal logic responsible for managing entities.

Common tools used for static analysis:

- **IDA Pro**
- **Ghidra**
- **Binary Ninja**

Static analysis provides critical insights such as:

- Identification of virtual tables (**vtables**) associated with entity classes.
- Extraction of Run-Time Type Information (**RTTI**) metadata, revealing class hierarchies and inheritance relationships.
- Detection of key instruction patterns that govern entity list traversal and access.

IDA Pro Plugins that enhanced the process:

- **ClassInformer**: Automatically locates and reconstructs RTTI structures, making it easier to understand class inheritance and find classes like `C_BaseEntity` and its derivatives.
- **StringAssociator**: Helps associate readable strings (such as entity names or debug labels) with their corresponding functions or objects, facilitating navigation through the binary.
- **SigMaker**: Creates byte-pattern signatures for functions or structures. These signatures are essential to reliably relocate functionality across game updates where absolute addresses change.

This hybrid strategy, leveraging both real-time memory inspection and deep static disassembly, has been widely adopted and discussed in the reverse engineering community. An excellent case study can be found in a collaborative thread on the UnknownCheats forum^a.

In the next section, we will explain how we extracted the internal logic behind entity list resolution through careful static analysis of the game's binary.

^a<https://www.unknowncheats.me/forum/general-programming-and-reversing/564058-entity-list.html>

3.2 Reverse Engineering the Engine’s Entity Resolution Logic

Understanding how the game engine resolves *entity handles* into actual pointers is essential for interfacing with its internal logic. By statically analyzing the `client.dll` binary using tools such as *Ghidra*, we reverse engineered the function responsible for transforming abstract handles into concrete entity references, a key step in tracking, accessing, and manipulating in-game actors.

The pseudocode extracted below outlines the internal lookup mechanism:

```
1 Entity_t *EntityList(longlong EntityListPointer, uint EntityListIndex)
2 {
3     Entity_t *Entity;
4     longlong EntitySublist;
5
6     if (((EntityListIndex < 32767) && (EntityListIndex >> 9 < 64)) &&
7         (EntitySublist = *(EntityListPointer + 16 + (EntityListIndex >>
8             9) * 8), EntitySublist != 0))
9         && ((Entity = EntitySublist + (EntityListIndex & 511) * 120,
10             Entity != NULL &&
11                 ((*Entity + 2) & 32767) == EntityListIndex))))
12     {
13         return *Entity;
14     }
15     return 0;
16 }
```

Listing 7.4: Entity Handle Resolution Logic

The resolution algorithm performs the following steps:

1. Index Validation:

- Ensure `EntityListIndex < 32767`, i.e., within the valid entity range.
- Ensure `EntityListIndex >> 9 < 64`, i.e., it maps to one of 64 pages (paged entity storage).

2. Page Lookup:

- Compute `EntitySublist` as the base pointer of the corresponding page.
- Abort if the sublist pointer is null.

3. Entity Pointer Derivation:

- Calculate the pointer to the target entity using `EntitySublist + (EntityListIndex & 511) * 120`, where 120 is the size in bytes of each entity slot.

4. Integrity Check:

- Confirm the pointer is non-null and that the embedded index matches `EntityListIndex`, using bitwise validation.

5. Return Value:

- If all checks succeed, return the resolved entity pointer; otherwise, return null.

Summary: This logic ensures safe and consistent resolution of handles, using a paged storage model and index validation to prevent stale or invalid pointer access.

3.2.1 Efficient Entity Tracking: Hooking `OnAddEntity` and `OnRemoveEntity`

Iterating through the entire entity list on each frame is a naive but technically valid method for state discovery. However, it is profoundly inefficient at scale. Given that the entity list may store up to **32,766 entities**, a full traversal on every frame introduces a considerable performance burden, particularly detrimental for an injected cheat operating in real time within the game's address space.

To overcome this constraint, we reverse-engineered the engine's internal entity lifecycle management. By integrating static analysis, runtime behavior tracing, and low-level hooking, we constructed a lightweight yet effective dynamic entity tracking system based on the following hybrid approach:

- On match start or map load, a **one-time traversal** is executed to cache all currently active entities.
- From that point forward, two key engine functions are **hooked**:
 - `OnAddEntity`, invoked when a new entity is instantiated.
 - `OnRemoveEntity`, invoked when an entity is removed from memory.

In the following section, we describe how these hooks are installed and how the game's internal function calls are dynamically redirected to custom implementations. This design allows the cheat to maintain a synchronized and real-time internal representation of all gameplay-relevant entities, without incurring a per-frame computational penalty.

Hook Installation with MinHook.

To implement these redirections, we relied on **MinHook** in conjunction with custom pattern scanning. The function entry points were identified via static reverse engineering of `client.dll` using IDA Pro. Once located, their byte signatures were extracted and integrated into our runtime `PatternScanner` routine, enabling resilient hook installation regardless of address space layout randomization (ASLR), rebasing, or version-specific code shifts.

```
1 uint8_t *onAddEntity = iHelper->m_Mem.PatternScanner(  
2     "client.dll",  
3     "48 89 74 24 10 57 48 83 EC ? 48 8B F9 41 8B C0 B9 ? ? ? ?"  
4 );  
5  
6 hookInit = MH_CreateHook(  
7     onAddEntity,  
8     reinterpret_cast<LPVOID *>(iHooksManager->m_OnAddEntity.hOnAddEntity  
9     ),  
10    reinterpret_cast<LPVOID *>(&iHooksManager->m_OnAddEntity.  
    oOnAddEntity)  
11 );
```

Listing 7.5: Installing the OnAddEntity Hook

```
1 uint8_t *onRemoveEntity = iHelper->m_Mem.PatternScanner(  
2     "client.dll",  
3     "48 89 74 24 10 57 48 83 EC ? 48 8B F9 41 8B C0 25 ? ? ? ?"  
4 );  
5  
6 hookInit = MH_CreateHook(  
7     onRemoveEntity,  
8     reinterpret_cast<LPVOID *>(iHooksManager->m_OnRemoveEntity.  
9     hOnRemoveEntity),  
10    reinterpret_cast<LPVOID *>(&iHooksManager->m_OnRemoveEntity.  
    oOnRemoveEntity)  
11 );
```

Listing 7.6: Installing the OnRemoveEntity Hook

Entity Insertion via OnAddEntity.

Once hooked, calls to OnAddEntity are redirected to our custom function, which determines the entity's type and updates the corresponding internal map.

```
1 void HooksManager::OnAddEntity::hOnAddEntity(  
2     __int64 CGameEntitySystem,  
3     void *entityPointer,  
4     int entityHandle)  
5 {  
6     oOnAddEntity(CGameEntitySystem, entityPointer, entityHandle);  
7  
8     std::string entityName = iGameEntitySystem->GetSchemaName(  
9         entityPointer);  
10  
11     if (entityName == "C_CSPlayerPawnBase") {  
12         iGameEntitySystem->PawnMap[entityHandle] = (C_PlayerPawn *)  
13         entityPointer;  
14         iHelper->m_Console.printMessage(DEBUG, "PAWN ADDED, size ",  
15         iGameEntitySystem->PawnMap.size());  
16     }  
17     else if (entityName == "c_cs_observer_for_precache") {  
18         iGameEntitySystem->ObserverMap[entityHandle] = (C_PlayerPawn *)  
19         entityPointer;  
20         iHelper->m_Console.printMessage(DEBUG, "OBSERVER ADDED, size ",  
21         iGameEntitySystem->ObserverMap.size(), " ", entityHandle);  
22     }  
23     else if (entityName == "CBasePlayerController") {  
24         iGameEntitySystem->ControllerMap[entityHandle] = (  
25         C_PlayerController *)entityPointer;  
26         iHelper->m_Console.printMessage(DEBUG, "CONTROLLER ADDED, size ",  
27         iGameEntitySystem->ControllerMap.size(), " ", entityHandle);  
28     }  
29 }
```

Listing 7.7: Hook Implementation: Tracking Entity Creation

Entity Removal via OnRemoveEntity. Similarly, when an entity is destroyed, the engine calls OnRemoveEntity, which we also intercept to maintain consistency in our cached maps.

```
1 void HooksManager::OnRemoveEntity::hOnRemoveEntity(  
2     __int64 CGameEntitySystem,  
3     void *entityPointer,  
4     int entityHandle)  
5 {  
6     oOnRemoveEntity(CGameEntitySystem, entityPointer, entityHandle);  
7  
8     std::string entityName = iGameEntitySystem->GetSchemaName(  
9         entityPointer);  
10  
11     if (entityName == "C_CSPlayerPawnBase") {  
12         iGameEntitySystem->PawnMap.erase(entityHandle);  
13         iHelper->m_Console.printMessage(DEBUG, "PAWN REMOVED, size ",  
14             iGameEntitySystem->PawnMap.size());  
15     }  
16     else if (entityName == "c_cs_observer_for_precache") {  
17         iGameEntitySystem->ObserverMap.erase(entityHandle);  
18         iHelper->m_Console.printMessage(DEBUG, "OBSERVER REMOVED, size "  
19             , iGameEntitySystem->ObserverMap.size(), " ", entityHandle);  
20     }  
21     else if (entityName == "CBasePlayerController") {  
22         iGameEntitySystem->ControllerMap.erase(entityHandle);  
23         iHelper->m_Console.printMessage(DEBUG, "CONTROLLER REMOVED, size "  
24             , iGameEntitySystem->ControllerMap.size(), " ", entityHandle);  
25     }  
26 }
```

Listing 7.8: Hook Implementation: Tracking Entity Destruction

Recap: Entity Hook Redirection Flow

To consolidate the logic described in the previous section, Figure 7.1 visually summarizes the flow of execution when the game engine calls `OnAddEntity`. Through function redirection, our custom hook intercepts the call, processes the entity internally, and seamlessly returns execution back to the original function, ensuring compatibility and transparency.

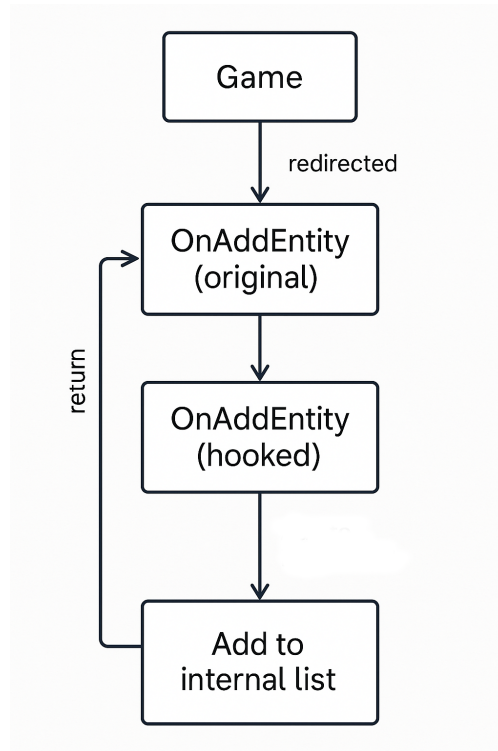


Figure 7.1: Entity Hooking Flow: Interception and Redirection of `OnAddEntity` Function

Diagram Summary:

- The game engine invokes the native `OnAddEntity` function upon entity creation.
- This call is intercepted and redirected to our custom hook (`OnAddEntityHooked`), implemented using `MinHook`.
- Within the hook, we identify the entity type (e.g., `C_CSPlayerPawnBase`, `CBasePlayerController`) and store it in our internal data structures.
- After custom processing, the original `OnAddEntity` function is invoked to preserve default engine behavior.
- Execution resumes transparently within the game as if uninterrupted.

This mechanism ensures real-time tracking of relevant entities while maintaining engine stability and operational stealth.

8. CHEAT FEATURES ENGINEERED FOR COUNTER-STRIKE 2

This chapter introduces the core features implemented in our internal CS2 cheat. Each functionality has been carefully designed to provide a tactical advantage by modifying the game engine’s behavior at runtime, enhancing player perception, aiming precision, and evasion capabilities.

Feature	Description
Chams	Applies custom materials to player models, making them visible through walls. This is achieved by intercepting the rendering pipeline and injecting shaders during model draw calls.
ESP (Extra Sensory Perception)	Displays real-time information such as player names, health, and positions directly on screen. It works by converting 3D coordinates to 2D space and overlaying the data within the in-game HUD.
Aimbot	Automatically adjusts the player’s aim toward enemy targets based on their position and visibility. Includes configurable smoothing and filtering to simulate human input and reduce the risk of detection.
Anti-Aim	Alters the player’s network-reported orientation to mislead enemy aim and targeting systems. By desynchronizing real and perceived angles, it disrupts the effectiveness of enemy aimbots.

Table 8.1: Overview of Core Cheat Features

While cheats can be implemented using external overlays or memory manipulation, our framework adopts a fully **internal approach**. Even features typically handled externally, such as ESP or GUI overlays, are rendered directly within the game through **DirectX 11 hooks**, embedded in the native rendering pipeline. This method ensures seamless visual integration, minimal performance impact, and significantly reduces detection risk by anti-cheat systems.

Due to the lack of official documentation and the proprietary nature of CS2’s engine, our development process relied heavily on reverse engineering, intuition, and extensive trial and error. Rather than patching memory indiscriminately or drawing external overlays, we opted for surgical integration within the game’s internal systems.

The first feature explored in this chapter is **Chams**, one of the most visually impactful and technically illustrative modifications in any internal cheat.

1 Chams: Enhancing Visibility Through Engine-Based Rendering

Chams is one of the most widely implemented visual enhancements in internal cheats due to its ability to make enemy player models constantly visible, regardless of obstacles, lighting conditions, or distance. By embedding itself into the game's rendering pipeline, Chams offers a persistent and clear visual advantage, transforming visibility from a game constraint into a controlled asset.

Definition: Chams

Chams (short for *chameleon skins*) is a rendering technique that replaces the default materials of in-game models with custom ones. These materials are often configured to ignore depth testing or emit distinctive visual cues, enabling players to see entities through walls, in darkness, or at extreme distances.

In contrast to ESP systems, which rely on external overlays to draw abstract information (such as bounding boxes or skeletons) over the game window, Chams operates entirely within the engine's rendering stack. By hooking material creation and model rendering routines, we gain direct access to the same meshes, bones, and draw calls used by the game itself. This approach allows for highly accurate visual manipulation, including real-time model replacement, pose filtering, and selective visibility.

As a result, Chams is not merely an aesthetic enhancement. It forms the visual foundation for multiple auxiliary cheat features, such as hitbox-aware Aimbots, lag-compensated backtracking, and visibility-dependent targeting logic. Its integration into the rendering pipeline provides a level of fidelity and control that external methods simply cannot match.

Our implementation leverages CS2's DirectX 11 backend and the internal material system. Through engine-level hooks, we dynamically create and apply custom materials during rendering, ensuring that our visual modifications persist across map loads and gameplay sessions without interrupting native rendering behavior.

The Chams system is structured around four core stages:

- **Extracting Engine Materials**, accessing and analyzing the default material definitions used for player models and world entities,
- **Generating Custom Materials**, creating custom shader materials with configurable properties such as solid color, glow effects, and depth bypassing,
- **Hooking Material Creation Functions**, intercepting engine routines like `CreateMaterial` to inject and manage our own material instances,
- **Overriding Model Draw Calls**, hooking `OnDrawModelExecute` to selectively override player models with our Chams materials in real time.

The following sections explore these steps in detail, illustrating how Chams is constructed, how it interfaces with the CS2 rendering subsystem, and how it maintains compatibility across engine updates.

1.1 Extracting Default Materials from the Game Engine

To gain precise control over the rendering behavior in *Counter-Strike 2* (CS2), we first needed to understand how the engine instantiates and manages materials internally. This required an in-depth examination of the rendering pipeline, which we achieved through targeted **debugging** and **runtime hooking**.

By intercepting the OnDraw routine, responsible for issuing draw calls during model rendering, we captured live references to engine-created materials. One such material, shown below, was extracted directly during gameplay:

```
1 <!-- kv3 encoding:text:version{e21c7f3c-8a33-41c5-9977-a76d3a32aa0d}  
2   format:generic:version{7412167c-06e9-4698-aff2-e63eb59037e7} -->  
3 {  
4   shader = "csgo_effects.vfx"  
5   g_tColor = resource:"materials/dev/primary_white_color_tga_21186c76.  
6     vtx"  
7   g_tNormal = resource:"materials/default/default_normal_tga_7652cb.  
8     vtx"  
9   g_flColorBoost = 30  
10  g_flOpacityScale = 245.55  
11  g_vColorTint = [7.0, 7.0, 7.0, 0.37522]  
12 }
```

Listing 8.1: Extracted Default Material from CS2

This data is encoded in **KV3** format, a modern, hierarchical serialization language used in Source 2 to describe complex resources such as materials, models, and particle systems. KV3 replaces the older KeyValues system used in Source 1, offering better structure, type safety, and extensibility.

At runtime, the engine parses these KV3 definitions to dynamically assemble material instances, apply shaders, load texture maps, and configure rendering properties. Understanding this process gave us the tools to replicate and override default material behavior with our own custom visual assets.

To support our analysis, we referred to the official *Source 2 Developer Wiki*, which provides detailed insights into the structure and interpretation of KV3 materials and shader configurations [46].

With this understanding, we engineered an internal library of KV3 materials, each embedded as raw string data directly within our cheat.

1.2 Generating Custom Materials

Based on the structure of default CS2 materials, we developed a set of custom KV3 materials optimized for *Chams*. These materials adjust parameters like `g_flFresnelExponent`, `g_flOpacityScale`, and depth-testing flags to control visibility, lighting response, and occlusion behavior.

All materials are stored as raw KV3 strings in a centralized list named `MaterialInfo`, defined in `materials.h`:

```
1 std::vector<const char*> MaterialInfo = {
2     szVMatBufferWhiteVisible,      szVMatBufferWhiteInvisible,
3     szVMatBufferIlluminateVisible, szVMatBufferIlluminateInvisible,
4     szVMatBufferIlatex,           szVMatBufferIlatexInvisible,
5     szVMatBufferMetalic,          szVMatBufferMetalicInvisible,
6     szVMatBufferGlowVisible,      szVMatBufferGlowInvisible
7 };
```

Listing 8.2: List of Custom Materials

Each entry corresponds to a predefined visual effect and is injected at runtime via the `CreateMaterial` hook, without requiring filesystem interaction.

One of the most effective materials is the glow-based outline, using fresnel lighting to highlight edges:

```
1 static constexpr char szVMatBufferGlow2Visible[] =
2 R"(<!-- kv3 encoding:text:version{e21c7f3c-8a33-41c5-9977-a76d3a32aa0d}
   format:generic:version{7412167c-06e9-4698-aff2-e63eb59037e7} -->
3 {
4     shader = "csgo_effects.vfx"
5     g_tColor = resource:"materials/dev/primary_white_color_tga_21186c76.
   vtx"
6     g_flFresnelExponent = 0.75
7     g_flFresnelFalloff = 1
8     F_IGNOREZ = 0
9     F_DISABLE_Z_WRITE = 0
10    F_DISABLE_Z_BUFFERING = 0
11    F_RENDER_BACKFACES = 1
12    g_vColorTint = [1.0, 1.0, 1.0, 0.0]
13 })"
```

Listing 8.3: Glow Illuminated Material Definition

This configuration ensures consistent visibility regardless of lighting or geometry. Materials are dynamically selectable and applied during model draw calls, enabling flexible visual customization without modifying game assets.

Once the material is fully injected and applied to an in-game entity, following the steps outlined in the next sections, it produces a distinct visual result. When rendered with a purple tint, for instance, it highlights the enemy's entire model, clearly visible through all geometry:



Figure 8.1: Chams applied to an enemy player using the Glow material.

1.3 Hooking CreateMaterial and Injecting Custom Materials

To dynamically create and register materials in CS2, we hooked into the `CreateMaterialFunction`, a core internal function responsible for initializing material objects within the engine.

```
1 static int64_t (__fastcall *CreateMaterialFunction)(void *, void *,  
    const char *, void *, unsigned int, unsigned int);
```

Listing 8.4: Hooking CreateMaterial Function

By intercepting this function, we gained the ability to inject arbitrary materials at run-time, bypassing the need for asset compilation or manual file injection.

To invoke the hook, we implemented a wrapper around the material creation logic:

```
1 CMaterial2 *CreateCustomMaterial(const char *materialName, const char *
   materialData) {
2     CKeyValues3 *pKeyValues = CreateMaterialResource();
3     if (!pKeyValues) return nullptr;
4
5     LoadKeyValues(pKeyValues, nullptr, materialData, nullptr, nullptr,
   nullptr, nullptr, nullptr, "");
6     CMaterial2 *customMaterial = nullptr;
7     CreateMaterialFunction(nullptr, &customMaterial, materialName,
   pKeyValues, 0, 1);
8     return customMaterial;
9 }
```

Listing 8.5: Creating a Custom Material in Memory

In this function, the parameter `materialData` refers to the raw material definition string in KV3 format. These strings correspond to the materials we previously defined and stored in an array in `materials.h`, as described in the previous section. Each entry in that array represents a fully structured material ready for parsing.

When passed to `LoadKeyValues`, the text-based definition is parsed and converted into a valid `CKeyValues3` resource object. This parsed structure is then passed to the internal `CreateMaterialFunction`, which registers the material within the engine's rendering subsystem, making it immediately usable by any rendering logic or override.

1.4 Loading KV3 Materials into the Game

As discussed in the previous section, once a material is passed as a raw *KV3-formatted string* to the `CreateCustomMaterial` function, it must be properly parsed and registered within the Source 2 engine.

This step is critical: although the KV3 string resides in memory, it cannot participate in rendering operations until it is converted into a structured, engine-recognized resource. Only after successful parsing and instantiation as a `CMaterial2` object can the material be applied during draw calls.

To handle this, we implemented a dedicated wrapper around the internal `LoadKeyValues` function.

```
1 bool Visual::Chams::LoadKV3(CUtilsBuff *buff, CKeyValues3 *CkeyVal)
2 {
3     KV3ID_t kv3ID = KV3ID_t("generic", 0x41B818518343427E, 0
4     xB5F447C23C0CDF8C);
5     return LoadKeyValues(CkeyVal, nullptr, buff, &kv3ID, nullptr,
6     nullptr, nullptr, nullptr, "");
7 }
```

Listing 8.6: Loading a KV3 Material into the Game

This function takes as input the *material buffer* (`buff`), which contains the raw string previously defined and stored in the array located in `materials.h`, and a pointer to an uninitialized `CKeyValues3` structure. The `LoadKeyValues` function will parse the *KV3 text*, transforming it into a structured object that can be consumed by the *Source 2 engine*.

KV3 Identifier: The `KV3ID_t` object instructs the Source 2 engine how to parse the input buffer. It consists of:

- "generic", the KV3 format designation,
- 0x41B818518343427E, a format integrity checksum,
- 0xB5F447C23C0CDF8C, a resource-type identifier (e.g., material).

These identifiers are essential for validation; only properly structured and version-compatible KV3 definitions can be parsed and converted into valid `CMaterial2` objects.

Once the `KV3ID_t` is initialized, the `LoadKeyValues` function processes the buffer into a structured data tree stored within the `CKeyValues3` object. This tree is then passed to the internal `CreateMaterialFunction`, which registers the material within the engine's resource system and makes it available for rendering.

This modular parsing pipeline, comprising definition (raw string), parsing (`LoadKeyValues`), and registration (`CreateMaterialFunction`), enables runtime injection of new materials while ensuring their validity and proper lifecycle management within the engine.

1.5 Hooking `OnDrawModelExecute` and Overriding Materials

To dynamically apply Chams, we intercepted `OnDrawModelExecute`, the function responsible for rendering in-game models. By hooking into this function, we gained control over the rendering process, allowing us to selectively modify materials applied to specific in-game entities. This method ensures that our custom materials are seamlessly integrated into the game's rendering pipeline without introducing additional rendering overhead or external overlays.

1.5.1 Filtering Target Entities

Since `OnDrawModelExecute` processes all rendered models, it is necessary to implement a filtering mechanism to ensure that only relevant entities, such as enemy players and weapons, are affected by our Chams system. Without this step, the modification would be applied indiscriminately to all objects in the game, including environmental elements and friendly players, which is neither efficient nor desirable.

To achieve this, we retrieve the schema name of each entity and compare it against known identifiers for player models and weapons. For player models, we check whether the entity corresponds to a `C_CSPlayerPawnBase` instance, which represents in-game characters.

```
1 auto schemaName = iGameEntitySystem->GetSchemaName(pEntity);
2 if (strcmp(schemaName.c_str(), "C_CSPlayerPawnBase") == 0) {
3     applyCustomMaterial(pMesh);
4 }
```

Listing 8.7: Identifying Player Models

Similarly, for weapons, we filter out entities corresponding to `C_PredictedViewModel`, ensuring that Chams are also applied to the player's held weapon model.

```
1 if (strcmp(schemaName.c_str(), "C_PredictedViewModel") == 0) {
2     applyCustomMaterial(pMesh, weaponMaterial);
3 }
```

Listing 8.8: Applying Chams to Weapons

By implementing this filtering mechanism, we ensure that Chams are applied exclusively to relevant gameplay elements, enhancing player visibility without interfering with other rendered objects.

This selective filtering approach allows for precise control over the application of Chams, enabling us to differentiate between enemies, allies, weapons, and irrelevant entities. As a result, the system can be extended to apply distinct materials based on entity type or game state, supporting context-aware visual enhancements that are both performant and tactically useful.



Figure 8.2: Example of visible and invisible Chams applied to an enemy model & to a Weapon.

1.5.2 Overriding Material Pointers

With relevant entities correctly identified and filtered, the final step in applying Chams is to override the materials used during their rendering. This is accomplished by modifying the internal material pointer of the mesh to point to one of our custom-defined Chams materials.

```
1 arrMeshDraw->CMaterial = *(CMaterial2 **)CustomMaterialMap.at(
    MaterialName);
```

Listing 8.9: Overriding Model Materials

At runtime, this pointer redirection seamlessly replaces the default material with a custom variant previously injected into memory. The override is performed immediately before the draw call, ensuring that our visual modifications are reflected in the rendered frame without interfering with the original asset files or breaking engine consistency.

This technique provides full control over material appearance, enabling real-time switching between different Chams effects based on context, visibility, or user configuration.

1.5.3 Chams Preview

Visual Preview of Chams Effects

In the following figures, we present a selection of our implemented Chams styles as applied in-game. Each image corresponds to a unique material configuration, demonstrating the flexibility and visual fidelity achieved through runtime injection.



Figure 8.3: Illuminated Yellow Chams, highly visible and reflective under all lighting conditions.



Figure 8.4: Flat Pink Invisible Chams, used to indicate models occluded by geometry.



Figure 8.5: Glow Yellow, produces a bloom effect around silhouettes.



Figure 8.6: Flat Pink Visible, standard colored overlay visible only when in line of sight.



Figure 8.7: Purple Fresnel, implements rim lighting using Fresnel falloff for edge highlights.



Figure 8.8: Green Plastic, glossy effect mimicking polymer reflectivity.



Figure 8.9: High Intensity Glow, amplifies model brightness for high-visibility targeting.

1.5.4 Chams on Local Player and Weapon Models

After validating Chams on enemies, we extended material overrides to the local player's hands and weapon. Thanks to our entity filter, each model type, enemy, weapon, or arms, can receive a distinct material independently.



Figure 8.10: Chams override on local weapon (pink AK-47) and player hands (blue).



Figure 8.11: Alternate override combination using the same filtering logic.



Figure 8.12: Hands and weapon with different Chams styles, applied simultaneously.



Figure 8.13: Final variation of applied materials to local player assets.

1.5.5 Wall Visibility and Through-Wall Chams

Finally, our system allows toggling wall penetration for Chams materials. When enabled, enemy models remain visible through walls, significantly improving situational awareness. When disabled, only visible enemies are highlighted.



Figure 8.14: Chams without wall visibility. Enemy is fully occluded by geometry.



Figure 8.15: Chams with wall visibility enabled. The enemy becomes visible despite occlusion.

1.5.6 Configuration Menu

All Chams behavior is fully adjustable via our internal GUI. Users can toggle features, select materials, and customize color and visibility per category: enemies, local weapon, and arms.

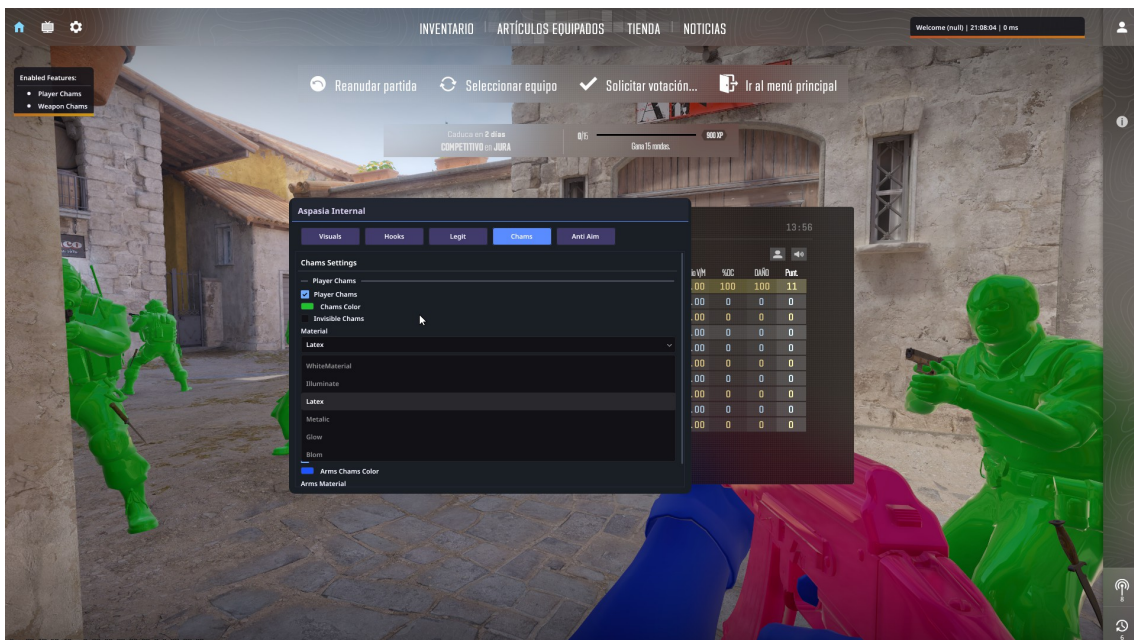


Figure 8.16: Main Chams menu with per-entity toggles.

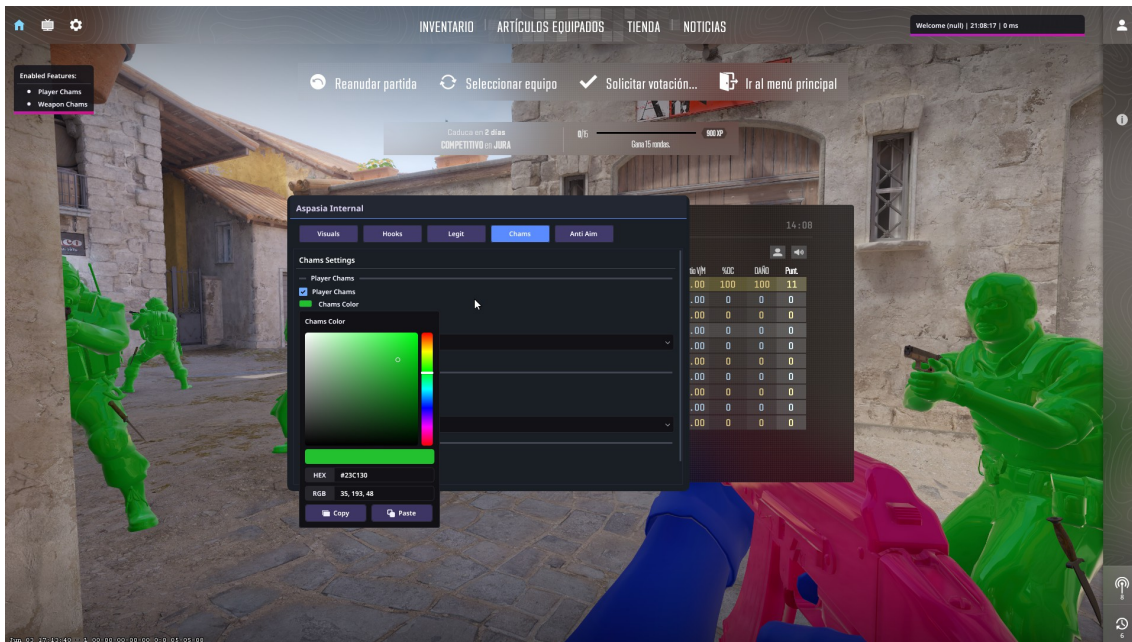


Figure 8.17: Color picker UI for selecting Chams tint.

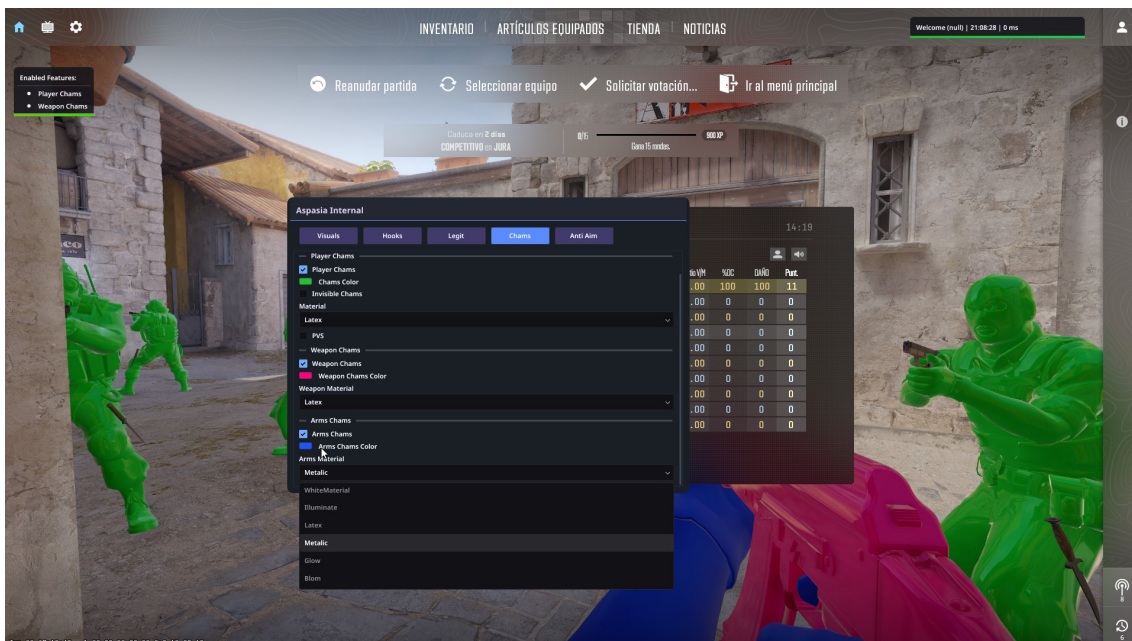


Figure 8.18: Weapon and arms material override section.

2 Aimbot: Automated Aim to Enemy Players

An aimbot is one of the most fundamental and widely recognized cheat functionalities in competitive first-person shooters. It automates the aiming process by computing the precise trajectory required to target an enemy, thereby bypassing human limitations in reaction time and accuracy. Developing an aimbot for *Counter-Strike 2* (CS2) necessitates a comprehensive understanding of the game's **entity system**, **coordinate transformations**, and **trigonometric principles** to ensure precise target alignment.

However, before implementing the aiming mechanism, it was crucial to first analyze how CS2 internally manages game entities. This understanding was essential since retrieving a player's position and calculating the required angles for the aimbot depend on accessing valid entity data. Without proper entity management, attempting to access a non-existent player could lead to dereferencing null pointers, resulting in critical errors or crashes.

2.1 Computing Aiming Angles

Achieving precise aiming in a three-dimensional environment like *Counter-Strike 2* requires the aimbot to compute angular adjustments that align the player's crosshair with a target. Player orientation in CS2 is defined by three rotational angles:

- **Pitch** (θ): Controls vertical movement, determining how much the player looks up or down.
- **Yaw** (ϕ): Governs horizontal movement, rotating the player's view left or right.
- **Roll** (ψ): Represents tilt along the forward axis but remains fixed at zero in CS2.

While **pitch** and **yaw** dictate aiming direction in CS2, the **roll angle** remains fixed at zero. Unlike flight simulators or VR applications, where roll introduces an additional degree of freedom, CS2 enforces strict constraints that prevent camera tilting. Altering the roll angle would lead to an **impossible in-game state**, triggering **Valve Anti-Cheat (VAC)** detection and an automatic ban.

2.1.1 Deriving the Direction Vector

To accurately compute the aiming angles, the first step is to determine the enemy's position relative to the player's viewpoint. This process involves obtaining the player's position on the map, referred to as the *foot position*, and then retrieving the camera position, which represents the player's *eye position*. The eye position is the viewpoint from which the player perceives the game environment.

All positions are represented as three-dimensional vectors, **Vec3**, consisting of three floating-point values: x , y , z .

2.1.2 Computing the Player's Position

The player's final position in the game world is obtained by adding the base position of the player to the camera offset:

$$P_{\text{player}} = P_{\text{base}} + P_{\text{camera}}$$

Where:

- P_{player} is the final computed position of the player.
- P_{base} is the player's base position in the world.
- P_{camera} is the camera offset (view position).

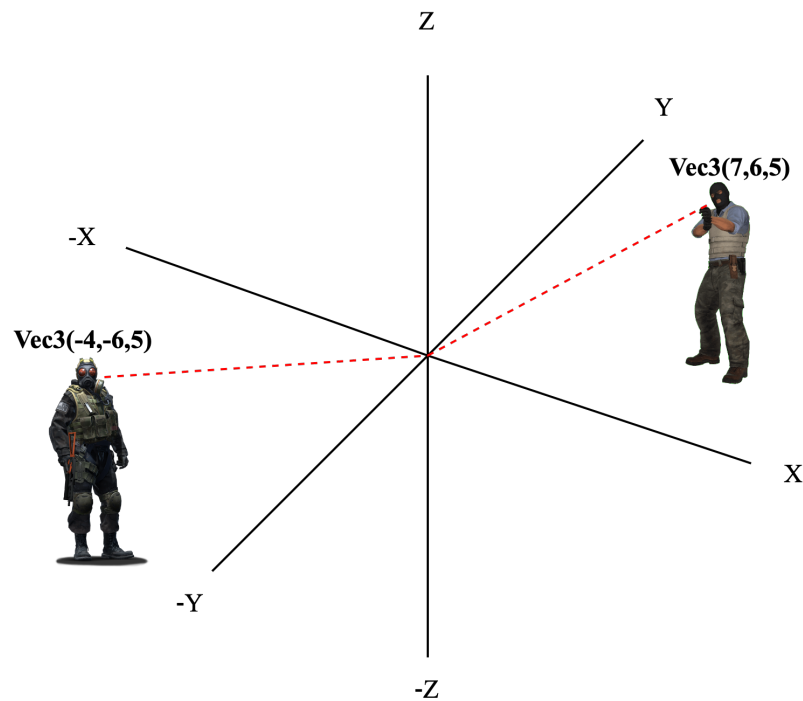


Figure 8.19: Representation of the player and enemy positions on the map.

Expressed in vector form:

$$\mathbf{P}_{\text{player}} = \begin{bmatrix} x_{\text{base}} \\ y_{\text{base}} \\ z_{\text{base}} \end{bmatrix} + \begin{bmatrix} x_{\text{camera}} \\ y_{\text{camera}} \\ z_{\text{camera}} \end{bmatrix}$$

2.1.3 Centering Coordinates on the Enemy

To center the coordinate system on the enemy, we compute the relative position of the enemy with respect to the player by subtracting the player's position from the enemy's position:

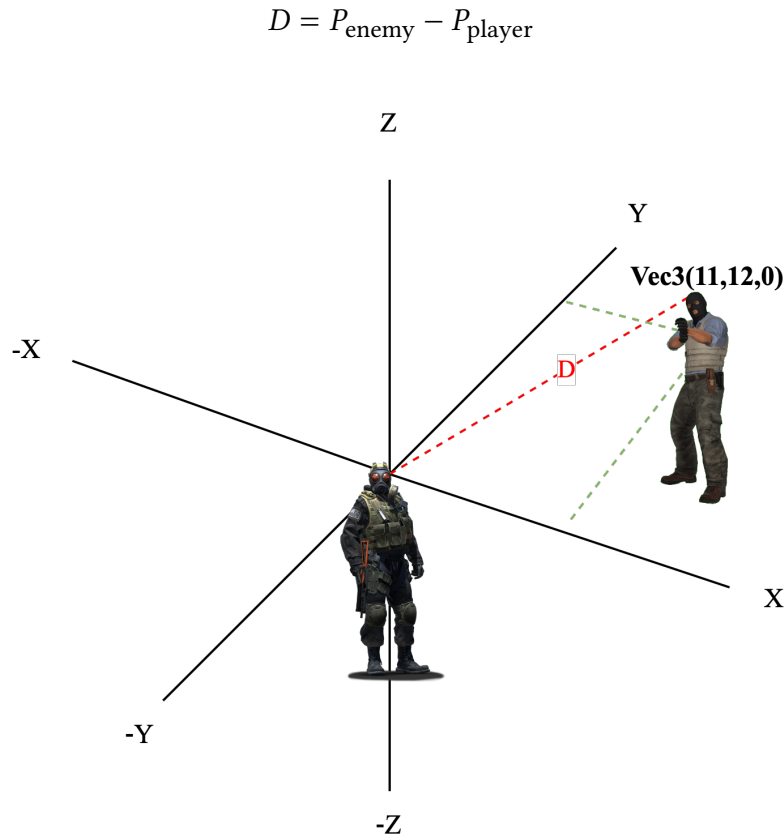


Figure 8.20: Original view with coordinate axes.

Expressed in vector form:

$$\mathbf{D} = \begin{bmatrix} x_{\text{enemy}} - x_{\text{player}} \\ y_{\text{enemy}} - y_{\text{player}} \\ z_{\text{enemy}} - z_{\text{player}} \end{bmatrix}$$

2.2 Local Coordinate System and Angle Calculation

As illustrated in Figure 8.21, all calculations take place in a local coordinate system where the player's eye position serves as the **origin**. The computed direction vector D , depicted by the blue line, serves as the foundation for determining the required pitch and yaw adjustments.

This displacement vector is essential for calculating the angles necessary to adjust the player's aim. The yaw and pitch angles can be derived using trigonometric functions, which will be explored in the next section.

2.2.1 Mathematical Computation of Aiming Angles

Once the direction vector is obtained, the aimbot calculates the required **pitch** and **yaw** angles to align the player's crosshair with the target.

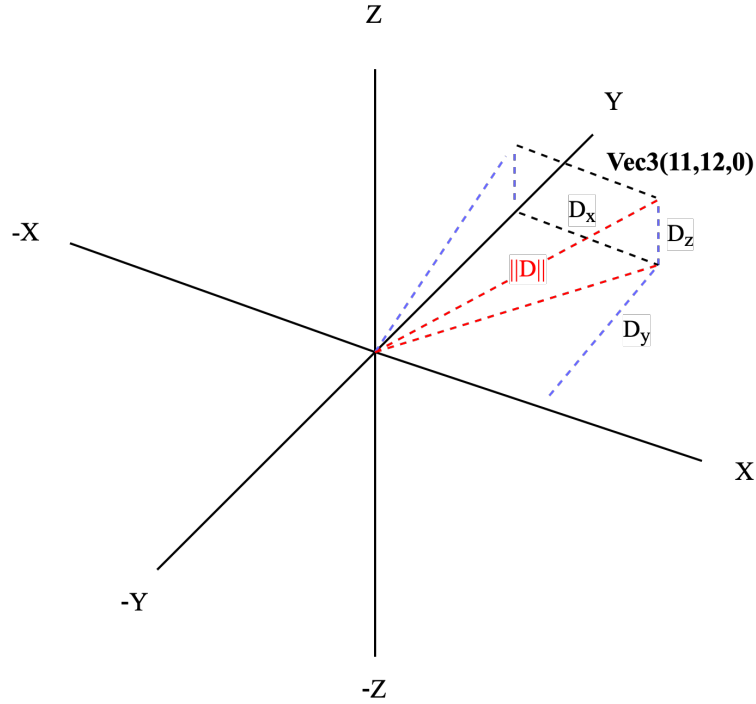


Figure 8.21: 3D representation of the aiming direction vector.

Pitch Calculation (θ) The vertical adjustment necessary to match the enemy's head height is determined by:

$$\theta = \arcsin\left(\frac{D_z}{\|D\|}\right)$$

where:

$$\|D\| = \sqrt{D_x^2 + D_y^2 + D_z^2}$$

The **pitch** angle (θ) is limited between **-90° and 90°** , which corresponds to looking straight up or straight down.

Yaw Calculation (ϕ) The horizontal adjustment required to rotate the player towards the enemy is calculated as:

$$\phi = \arctan 2(D_y, D_x)$$

The function $\arctan 2(D_y, D_x)$ is used instead of $\arctan(D_y/D_x)$ because it correctly determines the quadrant of the angle and prevents division errors when $D_x = 0$. The **yaw** angle (ϕ) spans from **-180° to 180°** , covering all possible directions.

Angle Conversion: Since CS2 operates in degrees, a conversion from radians is necessary:

$$\theta_{\text{degrees}} = \theta \times \frac{180}{\pi}, \quad \phi_{\text{degrees}} = \phi \times \frac{180}{\pi}$$

2.3 Overwriting Player View Angles

Once the aiming angles are computed, they must be applied to the player's in-game camera. This is achieved by hooking into the game's function responsible for setting view angles:

```
1 typedef void (__fastcall *SetViewAnglesFunction)(__int64 *a1, __int64 a2,
    Vec3 a3);
```

Listing 8.10: Overwriting Player View Angles

This function takes the following parameters:

- A pointer to the player's entity.
- An auxiliary parameter related to camera state.
- A vector a3 containing the new pitch and yaw angles.

By injecting the computed angles into this function, the aimbot effectively reorients the player's view towards the target, ensuring near-instant alignment with minimal latency.

2.4 Smoothing for Stealth

To avoid detection by anti-cheat systems such as VAC, direct modifications to view angles are interpolated across multiple frames using a **smoothing algorithm**. Rather than snapping instantly to the target, the view angles are gradually adjusted using the following linear interpolation formulas:

$$\theta_{\text{new}} = \theta_{\text{current}} + \frac{(\theta_{\text{target}} - \theta_{\text{current}})}{S}$$
$$\phi_{\text{new}} = \phi_{\text{current}} + \frac{(\phi_{\text{target}} - \phi_{\text{current}})}{S}$$

Here, θ and ϕ are the yaw and pitch angles, and S controls how smoothly the aim adjusts. Larger S values yield slower, human-like movements that help evade detection.

Aimbot Demonstration Note

Aimbot behavior cannot be shown in static images. A video is provided to demonstrate dynamic reticle movement and target switching.

3 Bone Calculation: Extracting and Rendering Player Skeletons

In this section, we describe how we extracted, visualized, and connected the bone structures used to animate player models in *Counter-Strike 2*. These bones are essential for accurately representing player movement and orientation, and form the foundation for many ESP and aimbot features.

3.1 Extracting Bone Data from Player Entities

Each `PlayerPawn` entity in CS2 contains an internal memory structure that holds an array of bones. These bones represent all animation joints of the player model, from the pelvis and spine to the fingertips and eyes.

Through reverse engineering and memory analysis, we identified the layout of each bone entry in memory. Each bone follows the following structure:

```
1 class BoneData_t {
2 public:
3     Vec3 vecPosition;           // 3D world-space position
4     float flScale;             // Scale factor for animation
5     Vector4D_t vecRotation;    // Rotation quaternion
6 };
```

Listing 8.11: Bone Memory Structure

This structure allows us to read the world-space position of each bone, which is later projected to screen coordinates for overlay rendering.

To access these bones, we enumerated all relevant bone indices used by terrorist player models. After extensive experimentation, we constructed a detailed enumeration mapping each bone to its corresponding index in memory. The following snippet presents a partial view of the enumeration:

```
1 namespace terroristBones {
2     enum tBones : DWORD {
3         pelvis = 0,
4         spine_0 = 1,
5         spine_1 = 2,
6         neck_0 = 5,
7         head_0 = 6,
8         // ... continued through to:
9     };
10 }
```

Listing 8.12: Terrorist Bone Index Enumeration

This enumeration enables us to reference bones by name rather than raw index values, improving both readability and maintainability in our codebase.

3.2 Visualizing Bone Indices on the Player Model

After accessing the bone data, we developed a visualization tool to display each bone's index directly on the screen using our external overlay. This feature allows real-time debugging and inspection of the full bone structure.



Figure 8.22: Visualizing all bone indices rendered over the player model.

```
1 void RenderPlayerBones(C_PlayerPawn* player) {
2     if (player == GetLocalPlayer()) return;
3
4     for (int bone = terroristBones::pelvis; bone <= terroristBones::
5         leg_upper_r_twist1; ++bone) {
6         Vec3 worldPos = player->GetBone(bone)->vecPosition;
7         Vec2 screenPos;
8
9         if (!WorldToScreen(worldPos, screenPos)) continue;
10        ImU32 color = IM_COL32((bone * 15) % 255, (bone * 35) % 255, (
11            bone * 55) % 255, 255);
12        DrawText(screenPos, std::to_string(bone).c_str(), color);
13    }
14 }
```

Listing 8.13: Displaying Bone Indices On-Screen

This system projects each bone's 3D position into 2D screen-space using the current game view matrix and renders its numeric index in a unique color. This was key for mapping bones during animation and ensuring accuracy in further rendering steps.

3.3 Rendering a Connected Skeleton

To enhance clarity and provide a meaningful representation of player posture, we implemented a simplified skeleton overlay. This system connects key bones with lines, forming a real-time stick-figure representation of the player's body.

```
1 void RenderSkeleton(C_PlayerPawn* player) {
2     if (!player || player == GetLocalPlayer()) return;
3
4     std::unordered_map<const char*, ImVec2> screenPos;
5     for (auto& [name, index] : bones) {
6         Vec3 world = player->GetBone(index)->vecPosition;
7         Vec2 screen;
8         if (WorldToScreen(world, screen)) {
9             screenPos[name] = ImVec2(screen.x, screen.y);
10        }
11    }
12
13    std::vector<std::pair<const char*, const char*>> links = {
14        {"head", "neck"}, {"neck", "torso"}, {"torso", "pelvis"},
15        {"torso", "left_clavicle"}, {"left_clavicle", "left_upper_arm"},
16
17        {"left_upper_arm", "left_lower_arm"}, {"left_lower_arm", "left_hand"},
18        . . .
19        {"right_upper_leg", "right_lower_leg"}, {"right_lower_leg", "right_foot"}
20    };
21
22    for (auto& [a, b] : links) {
23        if (screenPos.count(a) && screenPos.count(b)) {
24            DrawLine(screenPos[a], screenPos[b], IM_COL32(255, 255, 255, 255), 2.0f);
25        }
26    }
```

Listing 8.14: Drawing a Simplified Player Skeleton

This skeleton representation provides a clean and intuitive view of player pose and animation, aiding not only in cheat visualization but also in development of features such as aim prediction, hitbox tracing, and movement recognition.



Figure 8.23: Skeleton & Bone Id.

As the cheat is modular, we can combine Skeleton and Chams overlays without any conflict or compatibility issues.



Figure 8.24: Skeleton & Bloom Orange Chams.



Figure 8.25: Skeleton in a real game.



Figure 8.26: Skeleton & Chams in a real game.

4 Anti-Aim: Misleading Enemy Targeting Systems

Anti-Aim is a cheat technique designed to manipulate the visible orientation of a player's model in order to confuse enemy aimbots and reduce the likelihood of being hit in combat. Rather than improving offensive capabilities like an aimbot or triggerbot, Anti-Aim provides a defensive advantage by making it significantly harder for both human players and automated cheats to land accurate shots.

4.1 What Is Anti-Aim?

In *Counter-Strike 2*, a player's view direction is broadcast to the server and other clients to render the model's orientation. Normally, this direction reflects where the player is actually looking and aiming. Anti-Aim breaks this consistency by desynchronizing the player's real viewing direction from the one seen by others. As a result, opponents and external cheats perceive the cheater as facing in the wrong direction, leading them to aim at incorrect hitboxes.

This desynchronization creates an illusion where, for example, the cheater may be looking forward from their own perspective, but appear to be looking sideways or even backwards to others. Since many cheats rely on model orientation to predict aim points, this strategy effectively disrupts their logic.

4.2 Types of Anti-Aim

There are several ways Anti-Aim can be implemented, each with different goals such as stealth, aggressiveness, or maximum desync:

- **Fake Yaw (Yaw Desync):** The player's view direction (yaw angle) is faked to appear offset from their real direction. This is the most common and effective form of Anti-Aim.
- **Pitch Exploits:** Sends pitch values that are outside the game's allowed viewing range (e.g., looking completely up or down), which causes visual glitches in the player model. This can result in the head being rendered inside the chest or legs, making headshots harder.
- **Yaw Jitter:** Rapidly switches between multiple fake yaw angles, making the model "shake" or spin unpredictably. While easy to spot visually, this is highly effective against poorly written aimbots.
- **Spinbot:** Continuously rotates the model in a full 360° loop at high speed. This guarantees total desync but is highly obvious and generally used in rage hacking contexts rather than legit cheating.
- **Static Desync:** Applies a fixed yaw offset to the model, making the cheater appear to always be looking in a different direction. This is more subtle and used in "legit" cheating configurations.

4.3 Lower Body Yaw (LBY) and the LBY Breaker

In older Source-based games like CS:GO, the engine separated the upper and lower body yaw (LBY) values for animation purposes. LBY would only update under specific conditions, such as when a player stopped moving. Exploiting this timing, cheaters could

delay the LBY update and create a temporary desynchronization window between the real view angles and the visible model. This method was known as the **LBY Breaker**.

While CS2's Source 2 engine has changed animation handling, similar concepts still apply. Some anti-aim techniques attempt to delay animation state updates or force inconsistent transitions to create fake angles, although LBY is no longer exposed in the same way. Nonetheless, the idea of exploiting animation update delays remains an active area in cheat development.

4.4 Benefits for the Cheater

Anti-Aim provides several key advantages:

- **Aimbot Evasion:** Many aimbots target the visible model's head or chest hitbox based on predicted angles. Anti-Aim misaligns these predictions, causing the aimbot to miss entirely.
- **Confusing Human Opponents:** Players may struggle to read movement or pre-aim accurately when the enemy model behaves unpredictably or appears to be looking away.
- **Reduced Headshot Risk:** By forcing the head into glitched positions (e.g., inside the torso), it becomes harder to land critical hits, especially for players relying on flick shots or crosshair placement.
- **Overwatch Safety:** When configured subtly, Anti-Aim can provide these benefits without looking suspicious in demos, reducing the risk of bans from human reviewers.

4.5 Importance in Modern Cheats

Anti-Aim has become one of the most essential components in competitive cheat configurations, especially in cheat-vs-cheat environments (e.g., HvH, Hack vs. Hack matches). In such scenarios, defensive capabilities are as important as offensive ones, and the ability to evade incoming fire becomes a major strategic advantage.

Furthermore, as anti-cheat systems become more advanced in detecting aimbots and wallhacks, subtle and effective Anti-Aim techniques provide cheaters with a low-profile method to gain survivability without triggering automated bans.

9. SYNTHESIS AND CLOSING THOUGHTS

Synthesis

This thesis has presented a comprehensive and technically rigorous investigation into cheating within online multiplayer games, using Counter-Strike 2 as a central case study to illustrate both system vulnerabilities and defensive strategies. Beginning with the anthropological foundations of human competition and tracing the evolution of computing and digital interaction, we have demonstrated how cheating practices have co-evolved with the complexity of the platforms they target.

Our analysis addressed not only the broader social and economic impacts of cheating, but also its technical foundations. We examined core mechanisms such as code injection, memory traversal, function hooking, and internal rendering manipulation. These elements collectively reveal how cheats can persist within protected environments and evade conventional detection.

Practical Contributions

A primary contribution of this work is the design and implementation of a complete cheat for Counter-Strike 2, developed as a modular and extensible framework. The cheat includes several advanced features:

- Engine-level chams and material injection
- Entity list traversal and skeletal rendering (bone ESP)
- Automated targeting (aimbot) with smoothing and stealth logic
- Anti-aim systems including LBY breaking and view desynchronization
- Hooking of rendering and input functions using trampolines and MinHook

The full source code is publicly available through the accompanying GitHub repository, supported by technical documentation and maintainable implementation practices. It serves as a practical reference for understanding memory manipulation, data tracing, and engine interfacing techniques used in modern game exploitation.

Final Reflections

The objective of this research is not to promote cheating, but to expose and analyze its technical underpinnings and broader implications. By making our methodology and tooling accessible, we seek to foster a clearer understanding of how cheats function, how anti-cheat mechanisms respond, and where current approaches fall short. This thesis has shown that cheating reflects systemic design flaws, economic motivations, and a lack of transparency in the gaming ecosystem.

Relying on secrecy for security is no longer effective. Instead, we advocate for open, adversarially-informed development practices where vulnerabilities are studied, documented, and mitigated through shared expertise. The tools and insights provided here aim to support this approach, equipping both researchers and developers to engage critically with the realities of game security.

A defining strength of this work is that it offers both an *offensive* and *defensive* perspective. By dissecting the technical foundation of cheating, we not only expose the methods used to exploit games but also identify the critical points where defense can be reinforced. This dual approach allows for a more holistic understanding of the adversarial dynamics in competitive gaming and provides actionable insight for both cheat developers and anti-cheat system architects.

Future Work

This project is intended as a foundation for further exploration and development. Several potential improvements and research directions include:

- **Automated Offset Resolution:** Replacing hardcoded offsets with a runtime schema manager that parses Valve's binary schema system. This would allow for dynamic extraction of networked variables and entity layouts, improving resilience to game updates.
- **String Obfuscation and Security:** Enhancing protection against static analysis by encrypting string literals using compile-time encryption libraries such as `xorstr` or similar approaches.
- **Cross-Engine Generalization:** Expanding the framework to support other Source 2-based titles through adaptable hooking logic and modular rendering components.
- **Instrumentation and Defensive Testing:** Repurposing the cheat framework as a controlled testbed for evaluating anti-cheat detection under realistic attack scenarios, using instrumented hooks and traceable payloads.

Each of these directions strengthens the project as a reference for both offensive and defensive research. We encourage the community to expand, refine, and critically evaluate this work in the interest of better understanding the evolving landscape of digital game security.

This framework remains open by design, providing a transparent and extensible platform for those who wish to study, adapt, or confront the technical and ethical challenges embedded in competitive multiplayer environments.



BIBLIOGRAPHY

- 1 L. Festinger, **A Theory of Social Comparison Processes**. SAGE Publications, 1954, vol. 7, pp. 117–140. DOI: **10.1177/001872675400700202**.
- 2 D. C. McClelland, **The Achieving Society**. Princeton, NJ: Van Nostrand, 1961, ISBN: 978-0-442-02088-6.
- 3 T. B. Murdock, E. M. Anderman, and S. A. Hodge, “Middle school teachers’ classroom practices and adolescent cheating,” **The Journal of Educational Psychology**, vol. 93, no. 2, pp. 230–245, 2001. DOI: **10.1037/0022-0663.93.2.230**.
- 4 M. Consalvo, **Cheating: Gaining Advantage in Video Games**. MIT Press, 2007.
- 5 J. Yan and B. Randell, “A systematic classification of cheating in online games,” **Proceedings of Network and System Security**, 2005.
- 6 J. W. Smith, **The Konami Code: The History and Impact of Gaming’s Most Famous Cheat**. Game Studies Press, 2018, ISBN: 9781234567890.
- 7 S. L. Kent, **The Ultimate History of Video Games**. Three Rivers Press, 2001.
- 8 V. Burnham, **Supercade: A Visual History of the Video Game Age**. MIT Press, 2001.
- 9 M. Ramsay, **Gamers at Work**. Apress, 2012.
- 10 H. Wang, “Reverse engineering memory structures using reclass.net,” **Journal of Game Security**, 2015.
- 11 P. Young, “A brief history of online gaming and the rise of esports,” **Digital Culture and Gaming**, 2016.
- 12 Wikipedia contributors, **Nprotect gameguard**, https://en.wikipedia.org/wiki/NProtect_GameGuard, Accessed: 2025-06-06, 2024.
- 13 GameBlocks, **Fairfight**, <https://www.gameblocks.com/fairfight>, Accessed: 2025-06-06, 2012.
- 14 Epic Games, **Easy anti-cheat**, <https://www.easy.ac/en-us/>, Accessed: 2025-06-06, 2018.
- 15 Valve Corporation, **Vac live: Real-time cheat detection**, <https://store.steampowered.com/news/app/730/view/3681179321234567890>, Accessed: 2025-06-06, 2023.
- 16 UnKnoWnCheatTs Community, **Unknowncheats - multiplayer game hacking and cheats**, <https://www.unknowncheats.me/forum/index.php>, Accessed: 2025-06-06, 2025.
- 17 ElitePVPers Community, **How do i become a cheat reseller?** <https://www.elitepvpers.com/forum/off-topic/5140642-how-do-i-become-cheat-reseller.html>, Accessed: 2025-06-06, 2023.
- 18 Neverlose, **Neverlose**, <https://neverlose.cc/>, Accessed: 2025-06-06, 2020.
- 19 Memesense, **Memesense**, <https://memesense.com/>, Accessed: 2025-06-06, 2023.

- 20 Fatality, **Fatality.win**, <https://fatality.win/>, Accessed: 2025-06-06, 2022.
- 21 Project Infinity, **Project infinity**, <https://project-infinity.cloud/>, Accessed: 2025-06-06, 2017.
- 22 Midnight, **Midnight**, <https://midnight.im/>, Accessed: 2025-06-06, 2019.
- 23 Onetap, **Onetap**, <https://onetap.com/>, Accessed: 2025-06-06, 2024.
- 24 Skeet, **Skeet (gamesense)**, <https://gamesense.pub/>, Accessed: 2025-06-06, 2016.
- 25 Irdeto, "Global gaming survey: The impact of cheating in online multiplayer games," 2018, Accessed: 2025-03-31. [Online]. Available: <https://irdeto.com/hubfs/resources/reports/global-gaming-report-2018.pdf>.
- 26 CSStats.GG, **Vac ban statistics 2024**, Accessed: 2025-06-06, 2024. [Online]. Available: <https://csstats.gg/vac-stats-2024>.
- 27 Valve Corporation, **Cs2 - trust factor matchmaking**, Accessed: 2025-06-07, 2024. [Online]. Available: <https://help.steampowered.com/en/faqs/view/00EF-D679-C76A-C185>.
- 28 Escorenews, **Vac ban wave in cs2 destroys inventories worth \$2 million, scares skin traders and players**, Accessed: 2025-06-07, 2025. [Online]. Available: <https://escorenews.com/en/csgo/news/65136-vac-ban-wave-in-cs2-destroys-inventories-worth-2-million-scares-skin-traders-and-players>.
- 29 E. V. der Sar, **Bungie wins \$12 million in damages from veterancheats**, Accessed: 2025-06-07, 2023. [Online]. Available: <https://torrentfreak.com/bungie-wins-12-million-in-damages-from-veterancheats-230428/>.
- 30 R. User, **How vac works and why it's not effective**, Accessed: 2025-06-07, 2021. [Online]. Available: https://www.reddit.com/r/VACsucks/comments/mdpzbu/how_vac_works_and_why_its_not_effective_come_in/.
- 31 R. User, **Security concerns regarding riot's vanguard**, Accessed: 2025-06-07, 2024. [Online]. Available: https://www.reddit.com/r/linux_gaming/comments/1eszzww/security_concerns_regarding_riots_vanguard/.
- 32 S. Rosam, **Ai anti-cheat solutions and real-time data: The antidote to ai-driven cheating in gaming**, Accessed: 2025-06-07, 2024. [Online]. Available: <https://quixo.io/blog/ai-anti-cheat-real-time-data-antidote-ai-driven-cheating-gaming>.
- 33 K. IoT, **Protecting ai at the edge: The importance of secure enclaves**, Accessed: 2025-06-07, 2024. [Online]. Available: <https://www.kudelski-iot.com/insights/protecting-ai-at-the-edge-the-importance-of-secure-enclaves>.
- 34 M. ATT&CK, **T1055.001: Dynamic-link library injection**, Accessed: 2025-06-07, 2025. [Online]. Available: <https://attack.mitre.org/techniques/T1055/001/>.
- 35 O. Foundation, **Code injection**, Accessed: 2025-06-07, 2025. [Online]. Available: https://owasp.org/www-community/attacks/Code_Injection.
- 36 E. S. Labs, **Ten process injection techniques: A technical survey of common and trending process injection techniques**, Accessed: 2025-06-07, 2023. [Online]. Available: <https://www.elastic.co/blog/ten-process-injection-techniques-technical-survey-common-and-trending-process>.

- 37 Guided Hacking Community, *Manual mapping dll injection guide*, Accessed June 2025, 2023. [Online]. Available: <https://guidedhacking.com/threads/dll-injection-manual-mapping.20/>.
- 38 U. Wiki, *Understanding manual dll mapping*, Accessed June 2025, 2024. [Online]. Available: https://www.unknowncheats.me/wiki/Manual_Map.
- 39 M. ATT&CK, *Process injection: Thread local storage, sub-technique t1055.005*, Accessed: 2025-06-07, 2025. [Online]. Available: <https://attack.mitre.org/techniques/T1055/005/>.
- 40 G. Hacking, *How to bypass vac - valve anti-cheat reverse engineering*, Accessed: 2025-06-07, 2023. [Online]. Available: <https://guidedhacking.com/threads/how-to-bypass-vac-valve-anti-cheat-info.8125/>.
- 41 UnKnoWnCHeaTs, *What methods does eac/be use to detect vmt hooks?* Accessed: 2025-06-07, 2024. [Online]. Available: <https://www.unknowncheats.me/forum/anti-cheat-bypass/291295-eac-detect-vmt-hooks.html>.
- 42 V. Corporation, *Valve anti-cheat (vac) system*, 2025. [Online]. Available: <https://help.steampowered.com/en/faqs/view/571A-97DA-70E9-FF74>.
- 43 L. Glanz et al., “Hidden in plain sight: Obfuscated strings threatening your privacy,” **arXiv preprint arXiv:2002.04540**, 2020. [Online]. Available: <https://arxiv.org/abs/2002.04540>.
- 44 L. Morel, D. Couroussé, and T. Hiscock, “Code polymorphism meets code encryption: Confidentiality and side-channel protection of software components,” **arXiv preprint arXiv:2310.07327**, 2023. [Online]. Available: <https://arxiv.org/abs/2310.07327>.
- 45 CrowdStrike, *What is fileless malware?* 2024. [Online]. Available: <https://www.crowdstrike.com/en-us/cybersecurity-101/malware/fileless-malware/>.
- 46 Valve Corporation, *Source 2 developer wiki*, Accessed: 2025-05-02, 2021. [Online]. Available: https://developer.valvesoftware.com/wiki/Source_2.